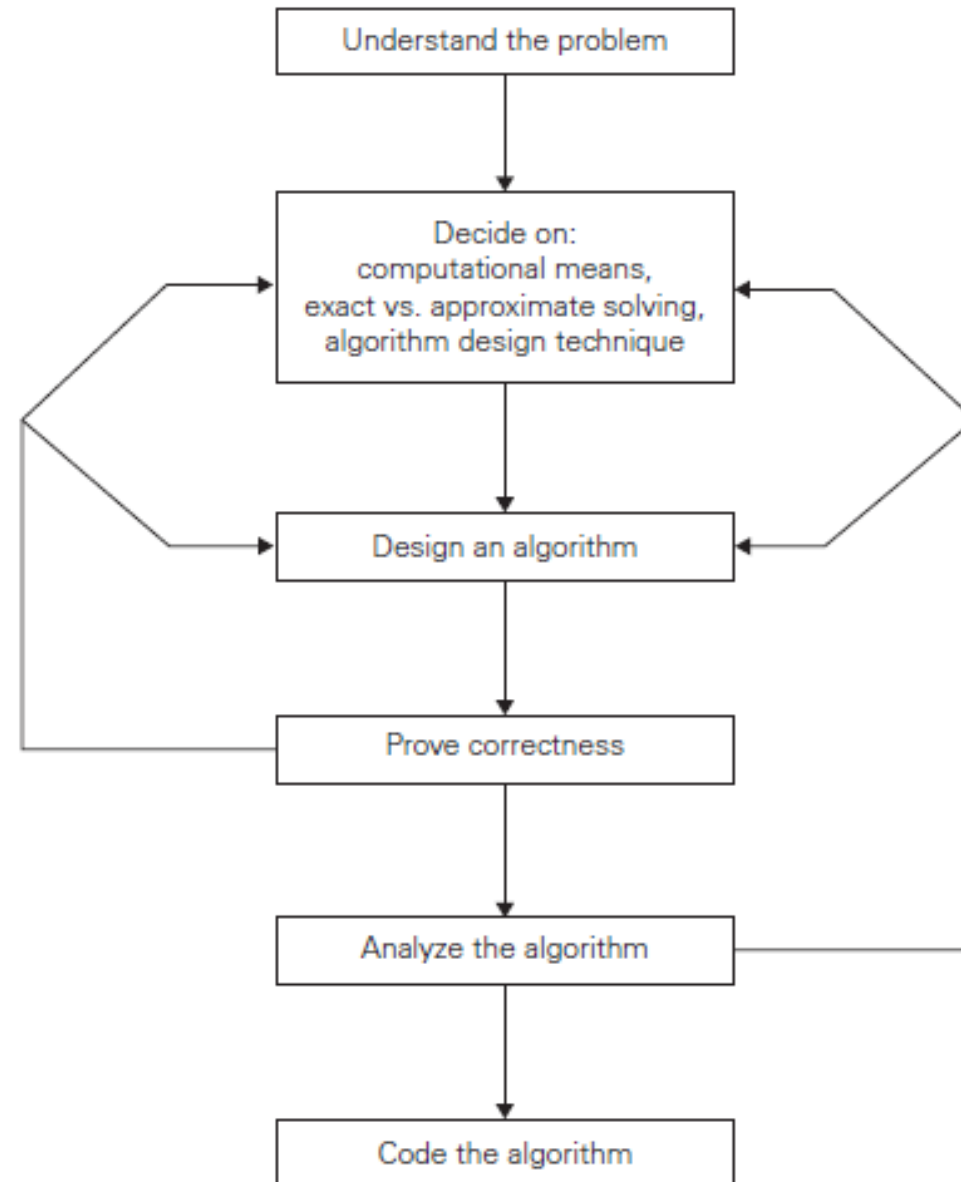




Euclid's algorithm for computing $\text{gcd}(m, n)$

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r .

Step 3 Assign the value of n to m and the value of r to n . Go to Step 1.

Alternatively, we can express the same algorithm in pseudocode:

ALGORITHM *Euclid*(m, n)

//Computes $\text{gcd}(m, n)$ by Euclid's algorithm

//Input: Two nonnegative, not-both-zero integers m and n

//Output: Greatest common divisor of m and n

while $n \neq 0$ **do**

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Middle-school procedure for computing $\gcd(m, n)$

Step 1 Find the prime factors of m .

Step 2 Find the prime factors of n .

Step 3 Identify all the common factors in the two prime expansions found in Step 1 and Step 2. (If p is a common factor occurring p_m and p_n times in m and n , respectively, it should be repeated $\min\{p_m, p_n\}$ times.)

Step 4 Compute the product of all the common factors and return it as the greatest common divisor of the numbers given.

Thus, for the numbers 60 and 24, we get

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\gcd(60, 24) = 2 \cdot 2 \cdot 3 = 12.$$

sieve of Eratosthenes

As an example, consider the application of the algorithm to finding the list of primes not exceeding $n = 25$:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3		5		7		9		11		13		15		17		19		21		23		25
2	3		5		7				11		13				17		19				23		25
2	3		5		7				11		13				17		19				23		

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the *regularity condition* $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

Consecutive integer checking algorithm for computing $\text{gcd}(m, n)$

Step 1 Assign the value of $\min\{m, n\}$ to t .

Step 2 Divide m by t . If the remainder of this division is 0, go to Step 3; otherwise, go to Step 4.

Step 3 Divide n by t . If the remainder of this division is 0, return the value of t as the answer and stop; otherwise, proceed to Step 4.

Step 4 Decrease the value of t by 1. Go to Step 2.

Asymptotic Growth

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Base switching:

$$\log_a n = \log_a b \log_b n$$

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element in A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

1.2-2

Suppose that for inputs of size n on a particular computer, insertion sort runs in $8n^2$ steps and merge sort runs in $64n \lg n$ steps. For which values of n does insertion sort beat merge sort?

1.2-3

What is the smallest value of n such that an algorithm whose running time is $100n^2$ runs faster than an algorithm whose running time is 2^n on the same machine?

1-1 Comparison of running times

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

	1 second	1 minute	1 hour	1 day	1 month	1 year	1 century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							

Efficiency

Different algorithms devised to solve the same problem often differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.

As an example, Chapter 2 introduces two algorithms for sorting. The first, known as *insertion sort*, takes time roughly equal to $c_1 n^2$ to sort n items, where c_1 is a constant that does not depend on n . That is, it takes time roughly proportional to n^2 . The second, *merge sort*, takes time roughly equal to $c_2 n \lg n$, where $\lg n$ stands for $\log_2 n$ and c_2 is another constant that also does not depend on n . Insertion sort typically has a smaller constant factor than merge sort, so that $c_1 < c_2$. We'll see that the constant factors can have far less of an impact on the running time than the dependence on the input size n . Let's write insertion sort's running time as $c_1 n \cdot n$ and merge sort's running time as $c_2 n \cdot \lg n$. Then we see that where insertion sort has a factor of n in its running time, merge sort has a factor of $\lg n$, which is much smaller. For example, when n is 1000, $\lg n$ is approximately 10, and when n is 1,000,000, $\lg n$ is approximately only 20. Although insertion sort usually runs faster than merge sort for small input sizes, once the input size n becomes large enough, merge sort's advantage of $\lg n$ versus n more than compensates for the difference in constant factors. No matter how much smaller c_1 is than c_2 , there is always a crossover point beyond which merge sort is faster.

For a concrete example, let us pit a faster computer (computer A) running insertion sort against a slower computer (computer B) running merge sort. They each must sort an array of 10 million numbers. (Although 10 million numbers might seem like a lot, if the numbers are eight-byte integers, then the input occupies about 80 megabytes, which fits in the memory of even an inexpensive laptop computer many times over.) Suppose that computer A executes 10 billion instructions per second (faster than any single sequential computer at the time of this writing) and computer B executes only 10 million instructions per second (much slower than most contemporary computers), so that computer A is 1000 times faster than computer B in raw computing power. To make the difference even more dramatic, suppose that the world's craftiest programmer codes insertion sort in machine language for computer A, and the resulting code requires $2n^2$ instructions to sort n numbers. Suppose further that just an average programmer implements merge sort, using a high-level language with an inefficient compiler, with the resulting code taking $50n \lg n$ instructions. To sort 10 million numbers, computer A takes

$$\frac{2 \cdot (10^7)^2 \text{ instructions}}{10^{10} \text{ instructions/second}} = 20,000 \text{ seconds (more than 5.5 hours) ,}$$

while computer B takes

$$\frac{50 \cdot 10^7 \lg 10^7 \text{ instructions}}{10^7 \text{ instructions/second}} \approx 1163 \text{ seconds (under 20 minutes) .}$$

By using an algorithm whose running time grows more slowly, even with a poor compiler, computer B runs more than 17 times faster than computer A! The advantage of merge sort is even more pronounced when sorting 100 million numbers: where insertion sort takes more than 23 days, merge sort takes under four hours. Although 100 million might seem like a large number, there are more than 100 million web searches every half hour, more than 100 million emails sent every minute, and some of the smallest galaxies (known as ultra-compact dwarf galaxies) contain about 100 million stars. In general, as the problem size increases, so does the relative advantage of merge sort.

INSERTION-SORT(A, n)

	<i>cost</i>	<i>times</i>
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 <i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

MERGE-SORT(A, p, r)

```
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                        // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                    // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
```

MERGE(A, p, q, r)

```
1   $n_L = q - p + 1$       // length of  $A[p : q]$ 
2   $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //    copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //    remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```


Here is the formal definition of O -notation. For a given function $g(n)$, we denote by $O(g(n))$ (pronounced “big-oh of g of n ” or sometimes just “oh of g of n ”) the *set of functions*

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .^1$$

$$“f(n) \in O(g(n))”$$

Just as O -notation provides an asymptotic *upper* bound on a function, Ω -notation provides an *asymptotic lower bound*. For a given function $g(n)$, we denote by $\Omega(g(n))$ (pronounced “big-omega of g of n ” or sometimes just “omega of g of n ”) the set of functions

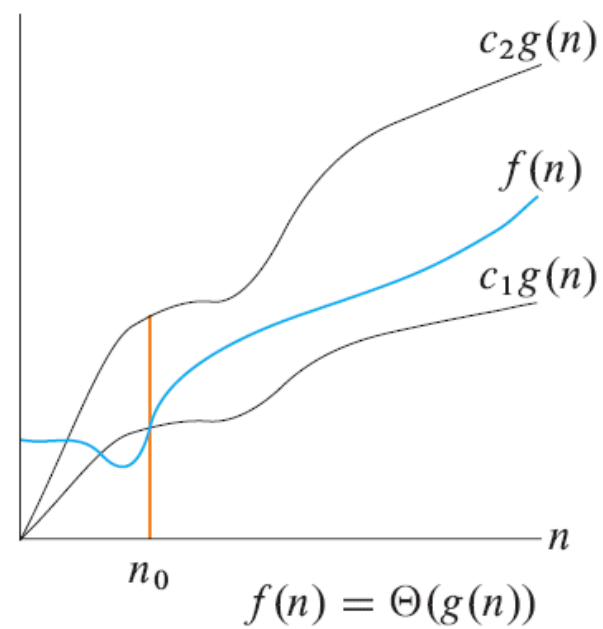
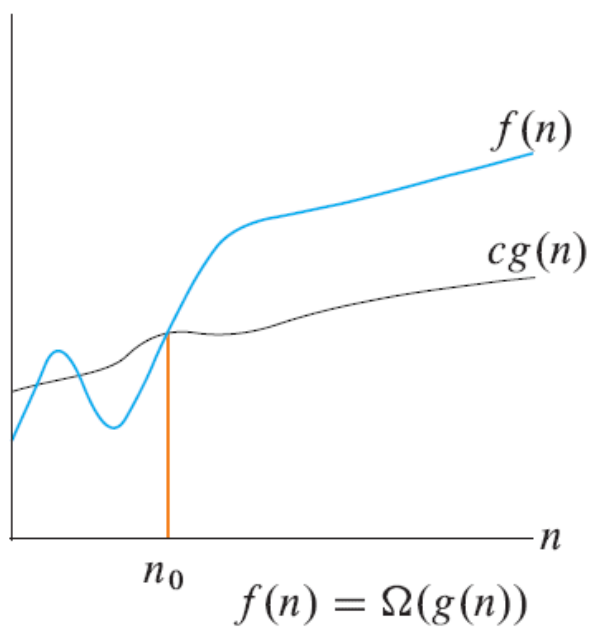
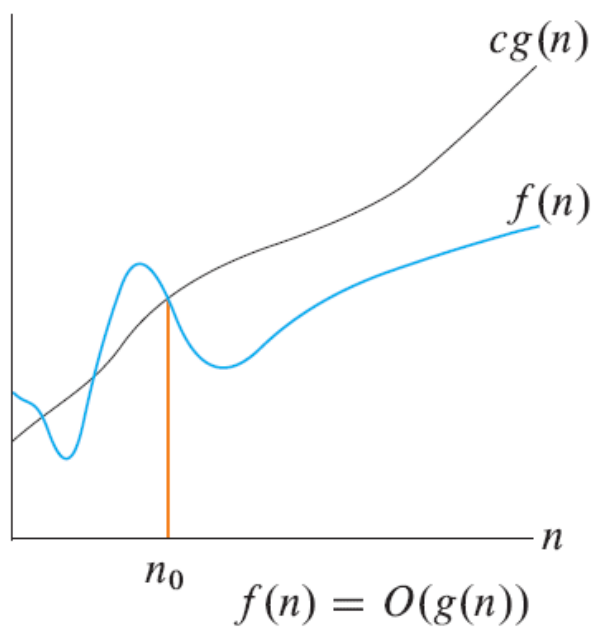
$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\} .$$

We use Θ -notation for *asymptotically tight bounds*. For a given function $g(n)$, we denote by $\Theta(g(n))$ (“theta of g of n ”) the set of functions

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}.$$

Theorem 3.1

For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. ■



Recapitulation of the Analysis Framework

- Both time and space efficiencies are measured as functions of the algorithm's input size.
- Time efficiency is measured by counting the number of times the algorithm's basic operation is executed. Space efficiency is measured by counting the number of extra memory units consumed by the algorithm.
- The efficiencies of some algorithms may differ significantly for inputs of the same size. For such algorithms, we need to distinguish between the worst-case, average-case, and best-case efficiencies.
- The framework's primary interest lies in the order of growth of the algorithm's running time (extra memory units consumed) as its input size goes to infinity.

ALGORITHM *MaxElement*($A[0..n - 1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n - 1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)
//Multiplies two square matrices of order n by the definition-based algorithm
//Input: Two $n \times n$ matrices A and B
//Output: Matrix $C = AB$
for $i \leftarrow 0$ **to** $n - 1$ **do**
 for $j \leftarrow 0$ **to** $n - 1$ **do**
 $C[i, j] \leftarrow 0.0$
 for $k \leftarrow 0$ **to** $n - 1$ **do**
 $C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$
return C

General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation. (As a rule, it is located in the inner-most loop.)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case efficiencies have to be investigated separately.
4. Set up a sum expressing the number of times the algorithm's basic operation is executed.⁴
5. Using standard formulas and rules of sum manipulation, either find a closed-form formula for the count or, at the very least, establish its order of growth.

$$\sum_{i=l}^u ca_i = c \sum_{i=l}^u a_i,$$

$$\sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i,$$

$$\sum_{i=l}^u 1 = u - l + 1 \quad \text{where } l \leq u \text{ are some lower and upper integer limits, } \mathbf{(S1)}$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2). \quad \mathbf{(S2)}$$

Note that the formula $\sum_{i=1}^{n-1} 1 = n - 1$, which we used in Example 1, is a special case of formula (S1) for $l = 1$ and $u = n - 1$.

$$n \in O(n^2), \quad 100n + 5 \in O(n^2), \quad \frac{1}{2}n(n-1) \in O(n^2).$$

$$n^3 \notin O(n^2), \quad 0.00001n^3 \notin O(n^2), \quad n^4 + n + 1 \notin O(n^2).$$

$$n^3 \in \Omega(n^2), \quad \frac{1}{2}n(n-1) \in \Omega(n^2), \quad \text{but } 100n + 5 \notin \Omega(n^2).$$

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

- 1.** Decide on a parameter (or parameters) indicating an input's size.
- 2.** Identify the algorithm's basic operation.
- 3.** Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.
- 4.** Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
- 5.** Solve the recurrence or, at least, ascertain the order of growth of its solution.

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

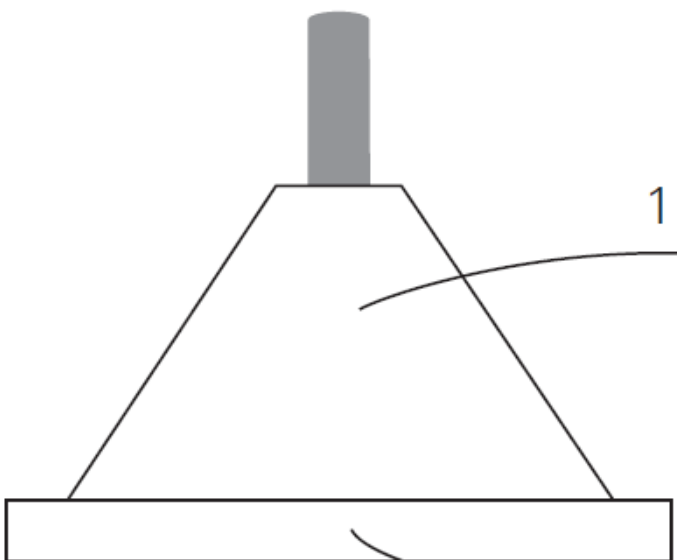
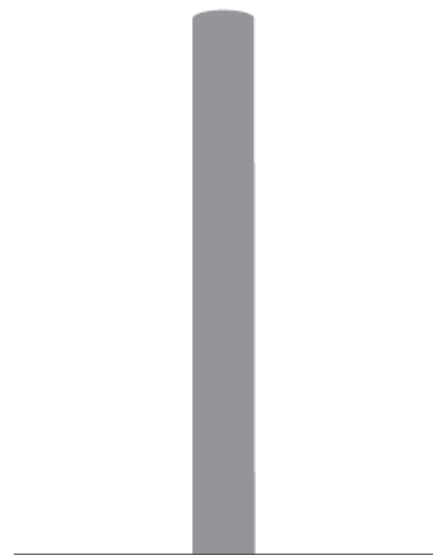
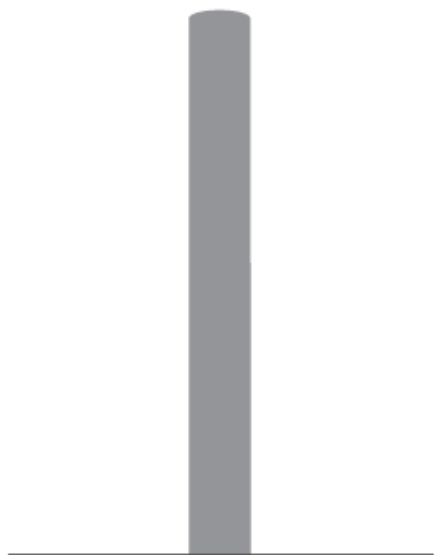
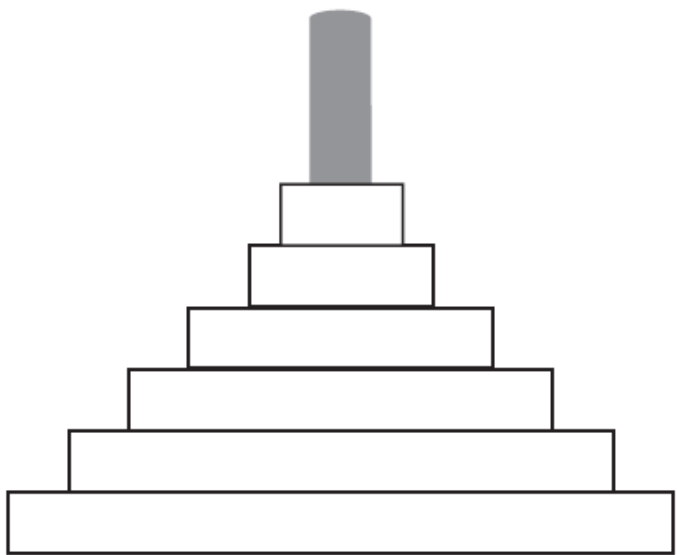
//Output: The value of $n!$

if $n = 0$ **return** 1

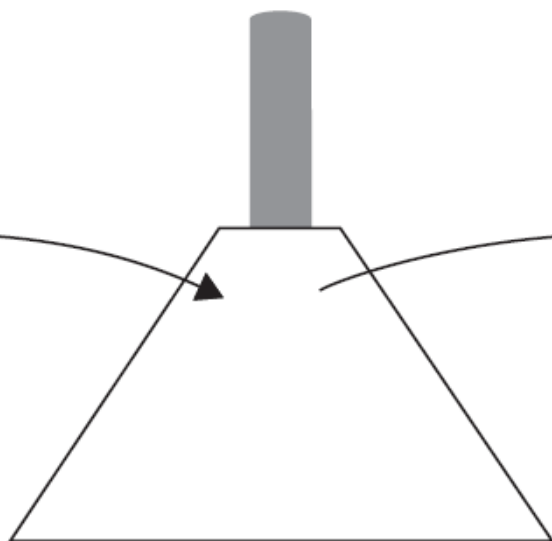
else return $F(n - 1) * n$

Tower of Hanoi puzzle.

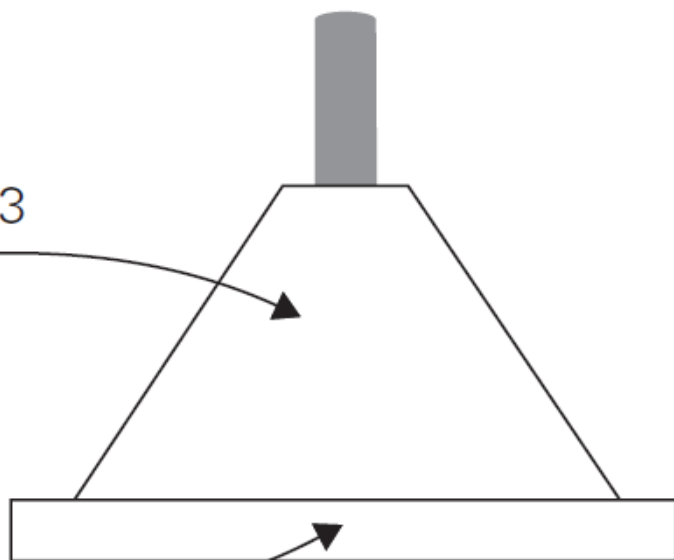
The problem has an elegant recursive solution, which is illustrated in Figure 2.4. To move $n > 1$ disks from peg 1 to peg 3 (with peg 2 as auxiliary), we first move recursively $n - 1$ disks from peg 1 to peg 2 (with peg 3 as auxiliary), then move the largest disk directly from peg 1 to peg 3, and, finally, move recursively $n - 1$ disks from peg 2 to peg 3 (using peg 1 as auxiliary). Of course, if $n = 1$, we simply move the single disk directly from the source peg to the destination peg.



1



3



```
def transfer(discs, source, destination, storage) :  
    if discs == 0 :  
        return  
    transfer(discs - 1, source, storage, destination)  
    print "Moving disc %d from %s to %s" % (discs, source, destination)  
    transfer(discs - 1, storage, destination, source)  
  
def main() :  
    n = int(raw_input("Enter the number of discs : "))  
    source = 'A'  
    destination = 'B'  
    storage = 'C'  
    transfer(n, source, destination, storage)  
if __name__=="__main__":  
    main()
```

ALGORITHM *Binary(n)*

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

$count \leftarrow 1$

while $n > 1$ **do**

$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

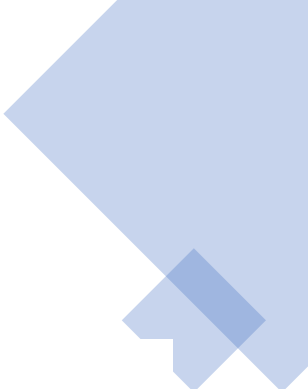
ALGORITHM *BinRec(n)*

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

else return $BinRec(\lfloor n/2 \rfloor) + 1$



1. Solve the following recurrence relations.

a. $x(n) = x(n - 1) + 5$ for $n > 1$, $x(1) = 0$

b. $x(n) = 3x(n - 1)$ for $n > 1$, $x(1) = 4$

c. $x(n) = x(n - 1) + n$ for $n > 0$, $x(0) = 0$

d. $x(n) = x(n/2) + n$ for $n > 1$, $x(1) = 1$ (solve for $n = 2^k$)

e. $x(n) = x(n/3) + 1$ for $n > 1$, $x(1) = 1$ (solve for $n = 3^k$)



- 
8. a. Design a recursive algorithm for computing 2^n for any nonnegative integer n that is based on the formula $2^n = 2^{n-1} + 2^{n-1}$.
- b. Set up a recurrence relation for the number of additions made by the algorithm and solve it.
- c. Draw a tree of recursive calls for this algorithm and count the number of calls made by the algorithm.
- d. Is it a good algorithm for solving this problem?
- 

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} .$$

MATRIX-MULTIPLY(A, B, C, n)

```
1  for  $i = 1$  to  $n$                                 // compute entries in each of  $n$  rows
2      for  $j = 1$  to  $n$                                 // compute  $n$  entries in row  $i$ 
3          for  $k = 1$  to  $n$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$     // add in another term of equation (4.1)
```

The divide step views each of the $n \times n$ matrices A , B , and C as four $n/2 \times n/2$ submatrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}. \quad (4.2)$$

Then we can write the matrix product as

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad (4.3)$$

$$= \begin{pmatrix} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{pmatrix}, \quad (4.4)$$

which corresponds to the equations

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}, \quad (4.5)$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}, \quad (4.6)$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}, \quad (4.7)$$

```
4      return
5  // Divide.
6  partition  $A$ ,  $B$ , and  $C$  into  $n/2 \times n/2$  submatrices
       $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22};$ 
      and  $C_{11}, C_{12}, C_{21}, C_{22};$  respectively
7  // Conquer.
8  MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{11}, C_{11}, n/2$ )
9  MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{12}, C_{12}, n/2$ )
10 MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{11}, C_{21}, n/2$ )
11 MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{12}, C_{22}, n/2$ )
12 MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{21}, C_{11}, n/2$ )
13 MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{22}, C_{12}, n/2$ )
14 MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{21}, C_{21}, n/2$ )
15 MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{22}, C_{22}, n/2$ )
```

$$\begin{aligned}
\begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix} &= \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} * \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} \\
&= \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix},
\end{aligned}$$

$$m_1 = (a_{00} + a_{11}) * (b_{00} + b_{11}),$$

$$m_2 = (a_{10} + a_{11}) * b_{00},$$

$$m_3 = a_{00} * (b_{01} - b_{11}),$$

$$m_4 = a_{11} * (b_{10} - b_{00}),$$

$$m_5 = (a_{00} + a_{01}) * b_{11},$$

$$m_6 = (a_{10} - a_{00}) * (b_{00} + b_{01}),$$

$$m_7 = (a_{01} - a_{11}) * (b_{10} + b_{11}).$$

ALGORITHM *Quicksort*($A[l..r]$)

//Sorts a subarray by quicksort

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right

// indices l and r

//Output: Subarray $A[l..r]$ sorted in nondecreasing order

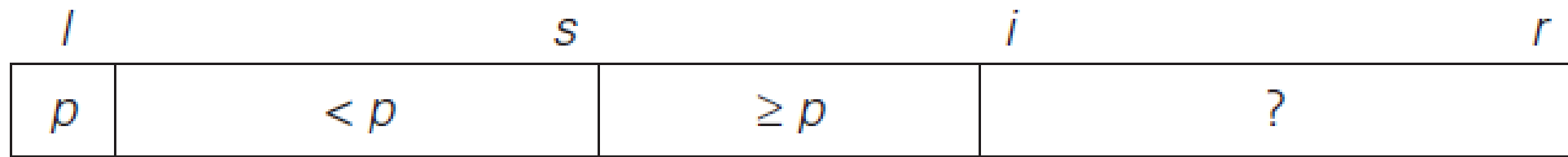
if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$ // s is a split position

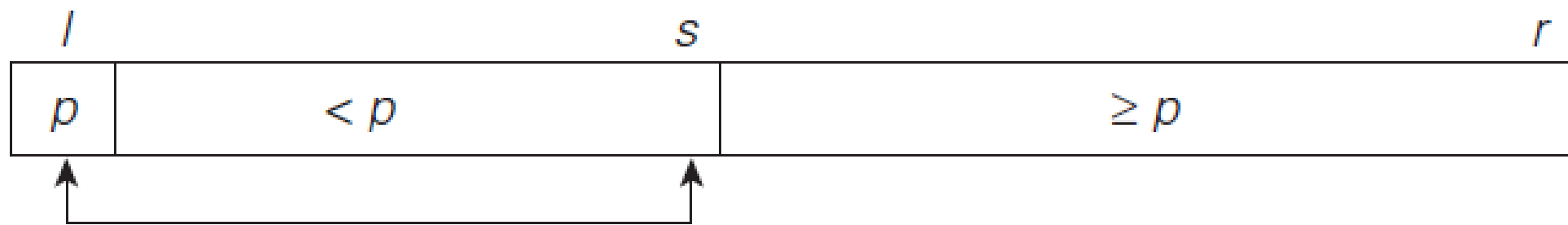
Quicksort($A[l..s - 1]$)

Quicksort($A[s + 1..r]$)

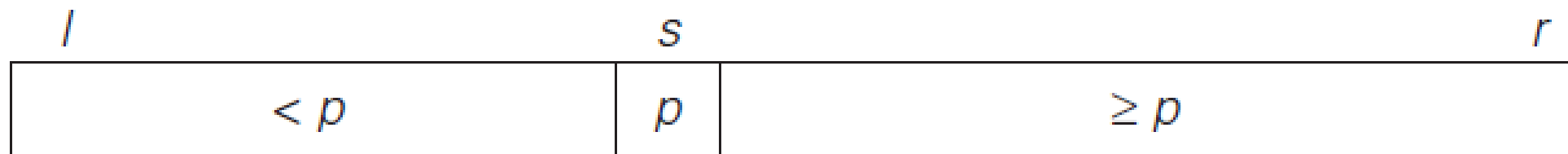
$$\underbrace{A[0] \dots A[s - 1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s + 1] \dots A[n - 1]}_{\text{all are } \geq A[s]}$$



(a)



(b)



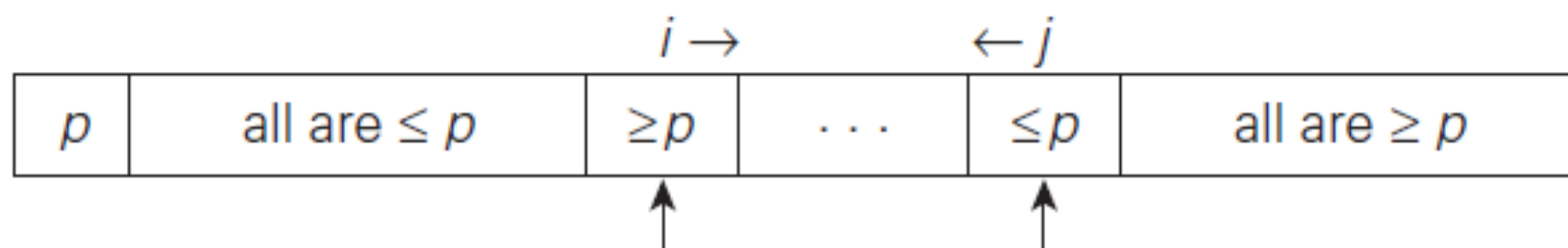
(c)

ALGORITHM *LomutoPartition*($A[l..r]$)

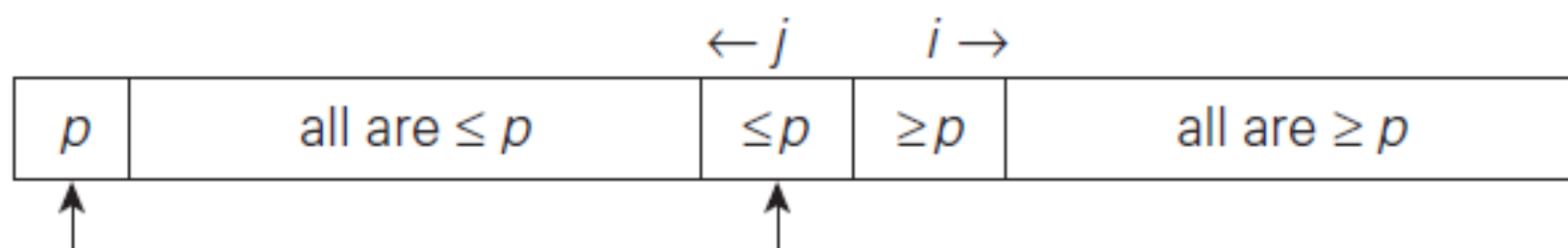
//Partitions subarray by Lomuto's algorithm using first element as pivot
//Input: A subarray $A[l..r]$ of array $A[0..n - 1]$, defined by its left and right
// indices l and r ($l \leq r$)
//Output: Partition of $A[l..r]$ and the new position of the pivot
 $p \leftarrow A[l]$
 $s \leftarrow l$
for $i \leftarrow l + 1$ **to** r **do**
 if $A[i] < p$
 $s \leftarrow s + 1$; swap($A[s], A[i]$)
swap($A[l], A[s]$)
return s



After both scans stop, three situations may arise, depending on whether or not the scanning indices have crossed. If scanning indices i and j have not crossed, i.e., $i < j$, we simply exchange $A[i]$ and $A[j]$ and resume the scans by incrementing i and decrementing j , respectively:



If the scanning indices have crossed over, i.e., $i > j$, we will have partitioned the subarray after exchanging the pivot with $A[j]$:



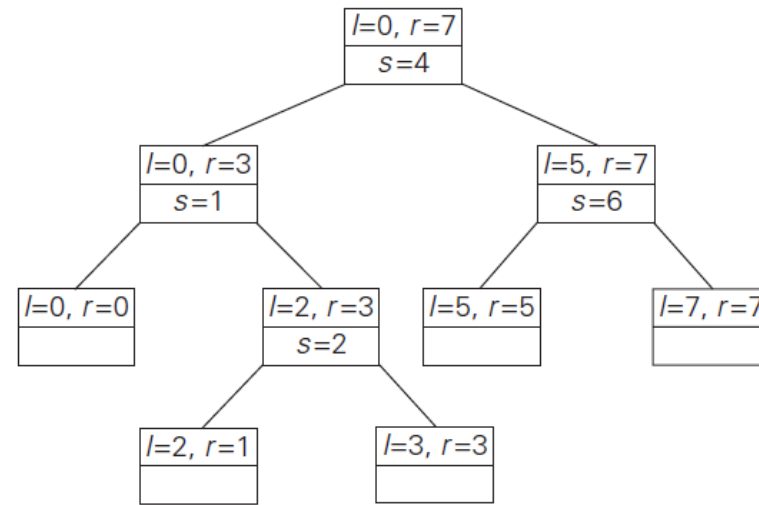
Finally, if the scanning indices stop while pointing to the same element, i.e., $i = j$,

ALGORITHM *HoarePartition*($A[l..r]$)

```
//Partitions a subarray by Hoare's algorithm, using the first element
//      as a pivot
//Input: Subarray of array  $A[0..n - 1]$ , defined by its left and right
//      indices  $l$  and  $r$  ( $l < r$ )
//Output: Partition of  $A[l..r]$ , with the split position returned as
//      this function's value
 $p \leftarrow A[l]$ 
 $i \leftarrow l; j \leftarrow r + 1$ 
repeat
    repeat  $i \leftarrow i + 1$  until  $A[i] \geq p$ 
    repeat  $j \leftarrow j - 1$  until  $A[j] \leq p$ 
    swap( $A[i], A[j]$ )
until  $i \geq j$ 
swap( $A[i], A[j]$ ) //undo last swap when  $i \geq j$ 
swap( $A[l], A[j]$ )
return  $j$ 
```

0	1	2	3	4	5	6	7
	<i>i</i>						<i>j</i>
5	3	1	9	8	2	4	7
5	3	1	9	8	2	4	7
5	3	1	4	8	2	9	7
5	3	1	4	8	2	9	7
5	3	1	4	2	8	9	7
5	3	1	4	2	8	9	7
2	3	1	4	5	8	9	7
2	<i>i</i>		<i>j</i>				
2	3	1	4				
2	<i>i</i>	<i>j</i>					
2	3	1	4				
2	<i>i</i>	<i>j</i>					
2	1	3	4				
2	<i>j</i>	<i>i</i>					
2	1	3	4				
1	2	3	4				
1							
		3	<i>ij</i>				
		<i>j</i>	4				
		3	4				
			4				

8 *i* *j*
8 9 7
8 7 9
8 *j* *i*
7 8 9
7 9



(b)

A first application of the brute-force approach often results in an algorithm that can be improved with a modest amount of effort.

Brute Force

ALGORITHM *SelectionSort*($A[0..n - 1]$)

//Sorts a given array by selection sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

$min \leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

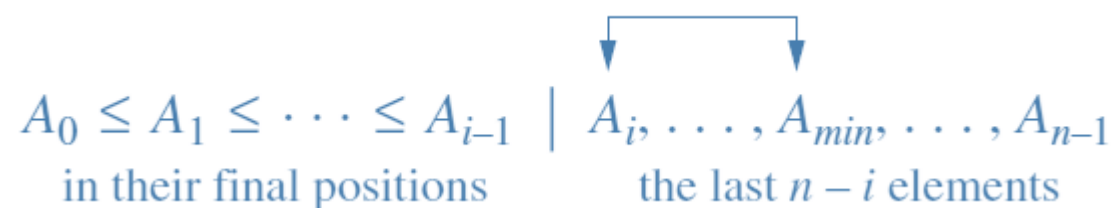
if $A[j] < A[min]$ $min \leftarrow j$

 swap $A[i]$ and $A[min]$

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n - 1 - i) = \frac{(n - 1)n}{2}.$$

selection sort is a $\Theta(n^2)$ algorithm on all inputs.

number of key swaps is only $\Theta(n)$, or, more precisely, $n - 1$



ALGORITHM *BubbleSort*($A[0..n - 1]$)

//Sorts a given array by bubble sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow 0$ **to** $n - 2 - i$ **do**

if $A[j + 1] < A[j]$ swap $A[j]$ and $A[j + 1]$

$A_0, \dots, A_j \overset{?}{\leftrightarrow} A_{j+1}, \dots, A_{n-i-1} \mid A_{n-i} \leq \dots \leq A_{n-1}$
in their final positions

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n - 2 - i) - 0 + 1]$$

$$= \sum_{i=0}^{n-2} (n - 1 - i) = \frac{(n - 1)n}{2} \in \Theta(n^2).$$

number of key swaps, however, depends on the input

worst case decreasing arrays, key swaps is the same as the number of key comparisons

$$S_{worst}(n) = C(n) = \frac{(n - 1)n}{2} \in \Theta(n^2).$$

ALGORITHM *SequentialSearch2*($A[0..n]$, K)

//Implements sequential search with a search key as a sentinel

//Input: An array A of n elements and a search key K

//Output: The index of the first element in $A[0..n - 1]$ whose value is

// equal to K or -1 if no such element is found

$A[n] \leftarrow K$

$i \leftarrow 0$

while $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

if a given list is known to be sorted: searching in such a list can be stopped as soon as an element greater than or equal to the search key is encountered.

ALGORITHM *BruteForceStringMatch*($T[0..n-1]$, $P[0..m-1]$)

//Implements brute-force string matching

//Input: An array $T[0..n-1]$ of n characters representing a text and

// an array $P[0..m-1]$ of m characters representing a pattern

//Output: The index of the first character in the text that starts a

// matching substring or -1 if the search is unsuccessful

for $i \leftarrow 0$ **to** $n - m$ **do**

$j \leftarrow 0$

while $j < m$ **and** $P[j] = T[i + j]$ **do**

$j \leftarrow j + 1$

if $j = m$ **return** i

return -1

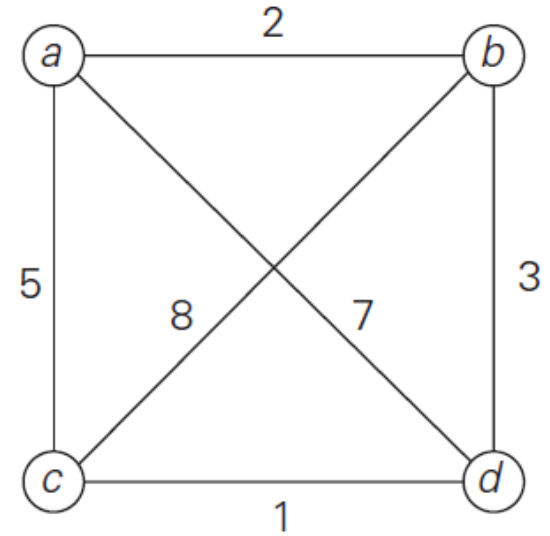
worst case, $O(nm)$ class.

searching in random texts $\Theta(n)$.

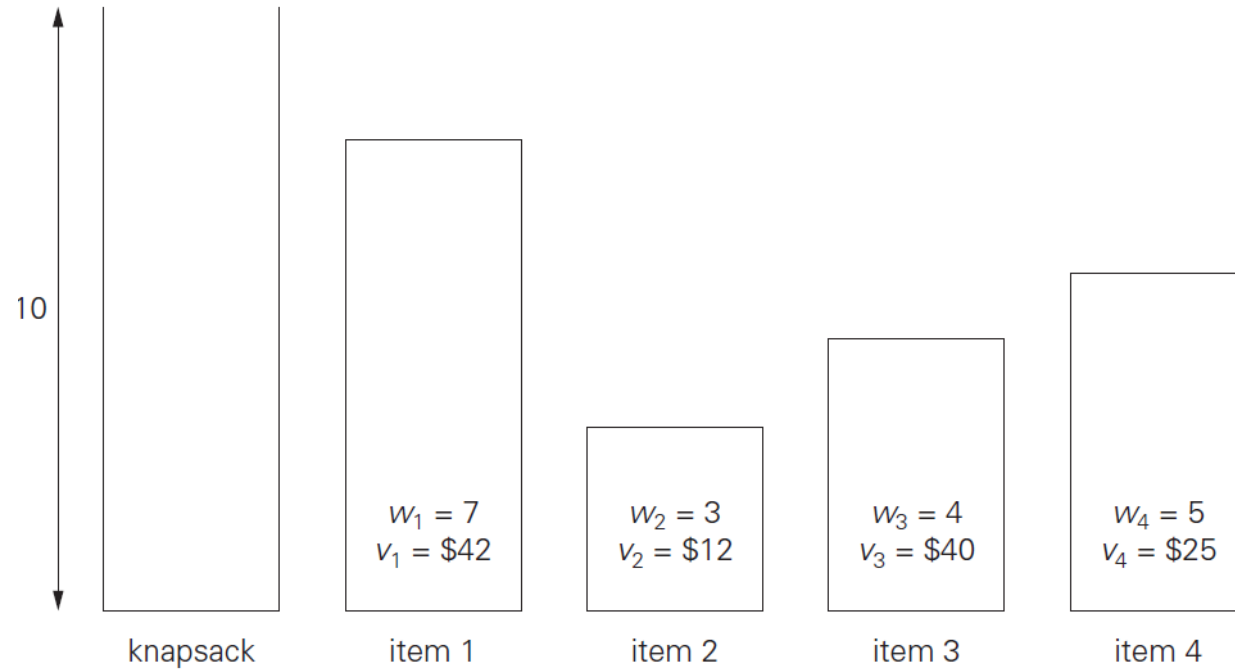
Exhaustive Search $\frac{1}{2}(n-1)!$

Traveling Salesman Problem

<u>Tour</u>	<u>Length</u>	
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$l = 2 + 8 + 1 + 7 = 18$	
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$l = 2 + 3 + 1 + 5 = 11$	optimal
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$l = 5 + 8 + 3 + 7 = 23$	
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$l = 5 + 1 + 3 + 2 = 11$	optimal
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$l = 7 + 3 + 8 + 5 = 23$	
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$l = 7 + 1 + 8 + 2 = 18$	



Knapsack Problem 2^n



Subset	Total weight	Total value
$\emptyset$	0	\$ 0
{1}	7	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1, 2}	10	\$54
{1, 3}	11	not feasible
{1, 4}	12	not feasible
{2, 3}	7	\$52
{2, 4}	8	\$37
{3, 4}	9	\$65
{1, 2, 3}	14	not feasible
{1, 2, 4}	15	not feasible
{1, 3, 4}	16	not feasible
{2, 3, 4}	12	not feasible
{1, 2, 3, 4}	19	not feasible

Assignment Problem $n!$

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

$\langle 1, 2, 3, 4 \rangle$ cost = $9 + 4 + 1 + 4 = 18$
 $\langle 1, 2, 4, 3 \rangle$ cost = $9 + 4 + 8 + 9 = 30$
 $\langle 1, 3, 2, 4 \rangle$ cost = $9 + 3 + 8 + 4 = 24$
 $\langle 1, 3, 4, 2 \rangle$ cost = $9 + 3 + 8 + 6 = 26$
 $\langle 1, 4, 2, 3 \rangle$ cost = $9 + 7 + 8 + 9 = 33$
 $\langle 1, 4, 3, 2 \rangle$ cost = $9 + 7 + 1 + 6 = 23$

Determine the number of character comparisons made by the brute-force algorithm in searching for the pattern GANDHI in the text

THERE_IS_MORE_TO_LIFE_THAN_INCREASING_ITS_SPEED

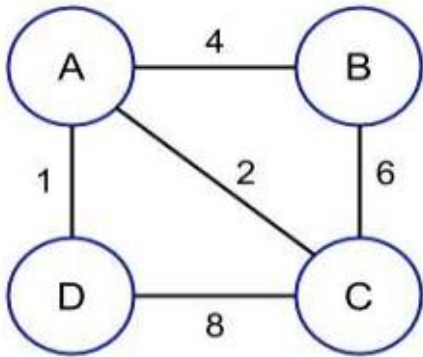


Knapsack problem

Given some items, pack the knapsack to get **the maximum total value**. Each item has some weight and some value. Total weight that we can carry is no more than some fixed number W . So we must consider weights of items as well as their values.

Item #(i)	Weight(w_i)	Value(b_i)
1	1	8
2	3	6
3	5	5

5. Find the solution to an instance of the travelling salesman problem by exhaustive search.



Assignment Problem

Hungarian Method - Example 1

	Job1	Job2	Job3	Job4
Crane1	4	2	5	7
Crane2	8	3	10	8
Crane3	12	5	4	5
Crane4	6	3	7	14

ALGORITHM *Backtrack*($X[1..i]$)

//Gives a template of a generic backtracking algorithm

//Input: $X[1..i]$ specifies first i promising components of a solution

//Output: All the tuples representing the problem's solutions

if $X[1..i]$ is a solution **write** $X[1..i]$

else //see Problem 9 in this section's exercises

for each element $x \in S_{i+1}$ consistent with $X[1..i]$ and the constraints **do**

$X[i + 1] \leftarrow x$

Backtrack($X[1..i + 1]$)

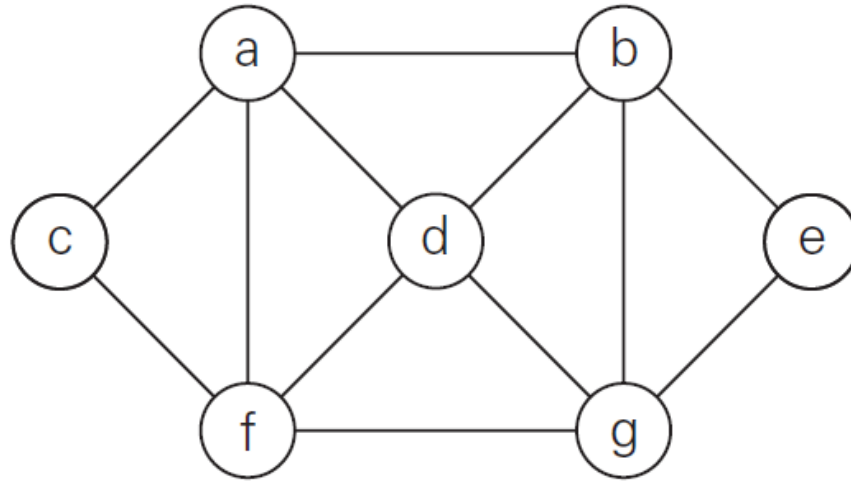
- A node in a state-space tree is said to be promising if it corresponds to a partially constructed solution that may still lead to a complete solution; otherwise, it is called non-promising.
- Leaves represent either non-promising dead end or complete solutions found by the algorithm.

Backtracking

- **Success in solving small instances of three difficult problems earlier in this section should not lead you to the false conclusion that backtracking is a very efficient technique. In the worst case, it may have to generate all possible candidates in an exponentially (or faster) growing state space of the problem at hand.**
- It is typically applied to difficult combinatorial problems for which no efficient algorithms for finding exact solutions possibly exist.
- Second, unlike the exhaustive search approach, which is doomed to be extremely slow for all instances of a problem, backtracking at least holds a hope for solving some instances of nontrivial sizes in an acceptable amount of time. This is especially true for optimization problems, for which the idea of backtracking can be further enhanced by evaluating the quality of partially constructed solutions.
- Third, even if backtracking does not eliminate any elements of a problem's state space and ends up generating all its elements, it provides a specific technique for doing so, which can be of value in its own.

Backtracking

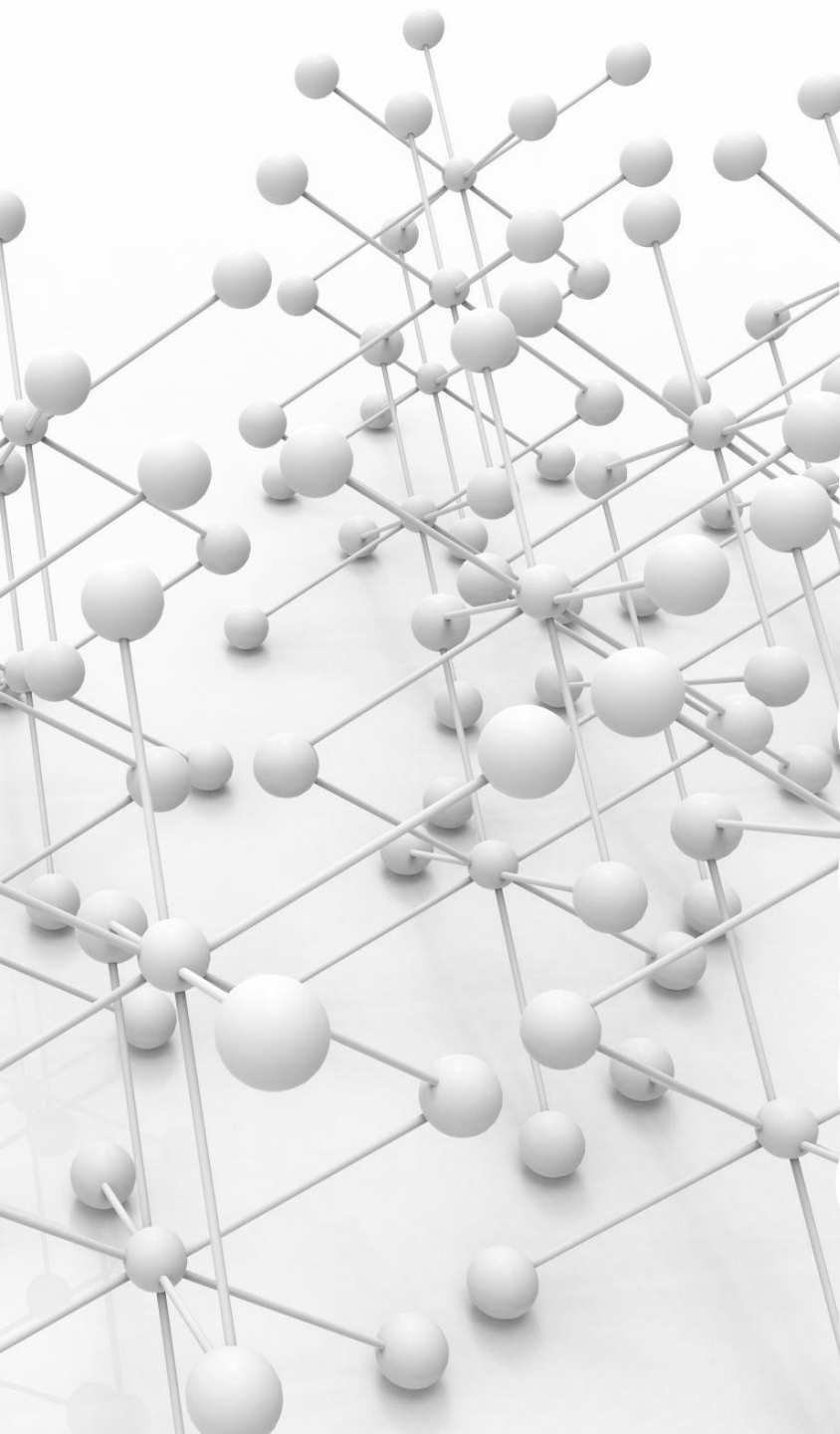
5. Apply backtracking to the problem of finding a Hamiltonian circuit in the following graph.



7. Generate all permutations of $\{1, 2, 3, 4\}$ by backtracking.
8. a. Apply backtracking to solve the following instance of the subset sum problem: $A = \{1, 3, 4, 5\}$ and $d = 11$.

Branch and Bound

- Central idea of **backtracking**: cut off a branch of the problem's state-space tree that cannot lead to a solution.
- An optimization problem usually subject to some constraint
 - seeks to minimize or maximize some objective function (a tour length, the value of items selected, the cost of an assignment etc.)
- **Standard terminology of optimization problems:**
 - A **feasible solution** is a point in the problem's search space that satisfies all the problem's constraints
 - e.g., a Hamiltonian circuit in the traveling salesman problem
 - or a subset of items whose total weight does not exceed the knapsack's capacity in the knapsack problem
 - An **optimal solution** is a feasible solution with the best value of the objective function
 - e.g., the shortest Hamiltonian circuit or the most valuable
 - subset of items that fit the knapsack



Principal idea of the branch-and-bound technique.

- Compared to backtracking, branch-and-bound requires two additional items:
 - a way to provide, for every node of a state-space tree, a bound on the best value of the objective function on any solution that can be obtained by adding further components to the partially constructed solution represented by the node
 - the value of the best solution seen so far
- If this information (Bound) is available, we can compare a node's bound value with the value of the best solution seen so far. If the bound value is not better than the value of the best solution seen so far—i.e., not smaller for a minimization problem and not larger for a maximization problem—the node is nonpromising and can be terminated (some people say the branch is “pruned”). Indeed, no solution obtained from it can yield a better solution than the one already available.

In general, we terminate a search path at the current node in a state-space tree of a branch-and-bound algorithm for any one of the following three reasons:

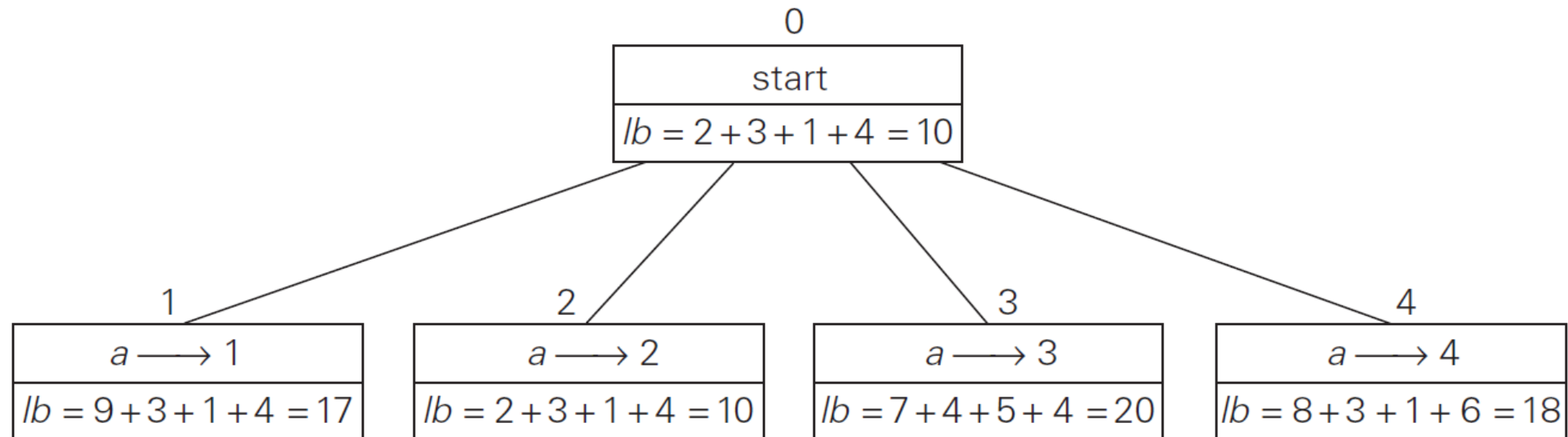
- The value of the node's bound is not better than the value of the best solution seen so far.
- The node represents no feasible solutions because the constraints of the problem are already violated.
- The subset of feasible solutions represented by the node consists of a single point (and hence no further choices can be made)—in this case, we compare the value of the objective function for this feasible solution with that of the best solution seen so far and update the latter with the former if the new solution is better.

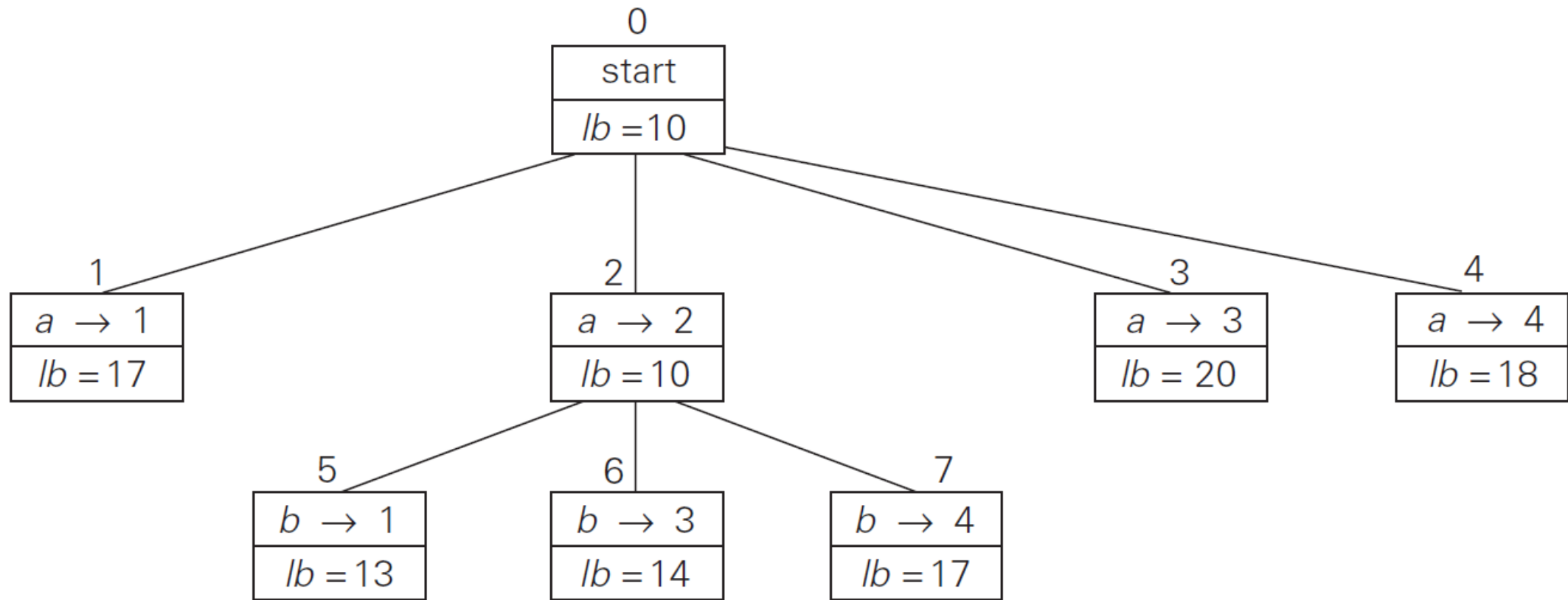
Assignment Problem

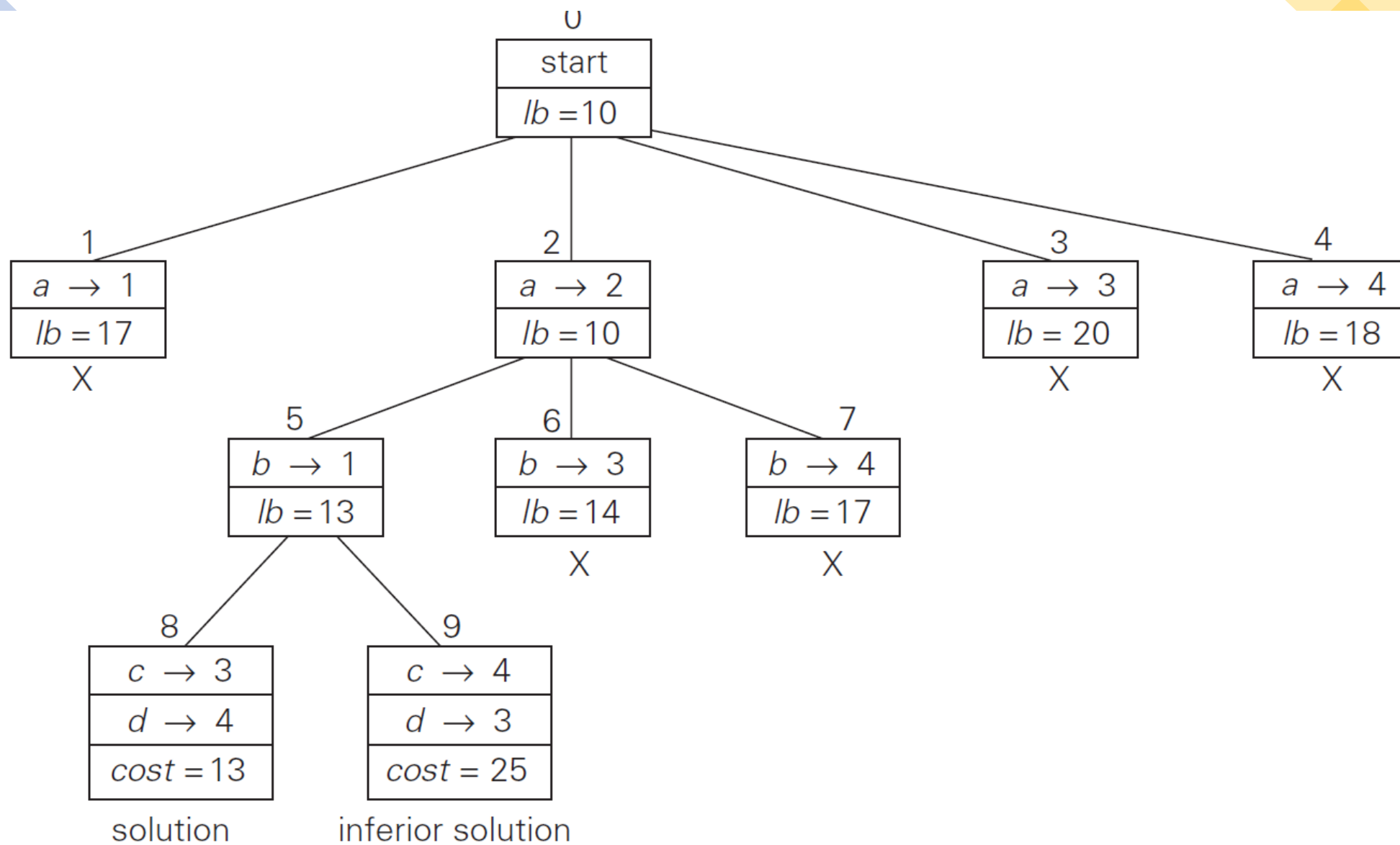
$$C = \begin{array}{cc} & \begin{array}{cccc} \text{job 1} & \text{job 2} & \text{job 3} & \text{job 4} \end{array} \\ \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix} & \begin{array}{l} \text{person } a \\ \text{person } b \\ \text{person } c \\ \text{person } d \end{array} \end{array}$$

Lower bound:

$$2 + 3 + 1 + 4 = 10.$$







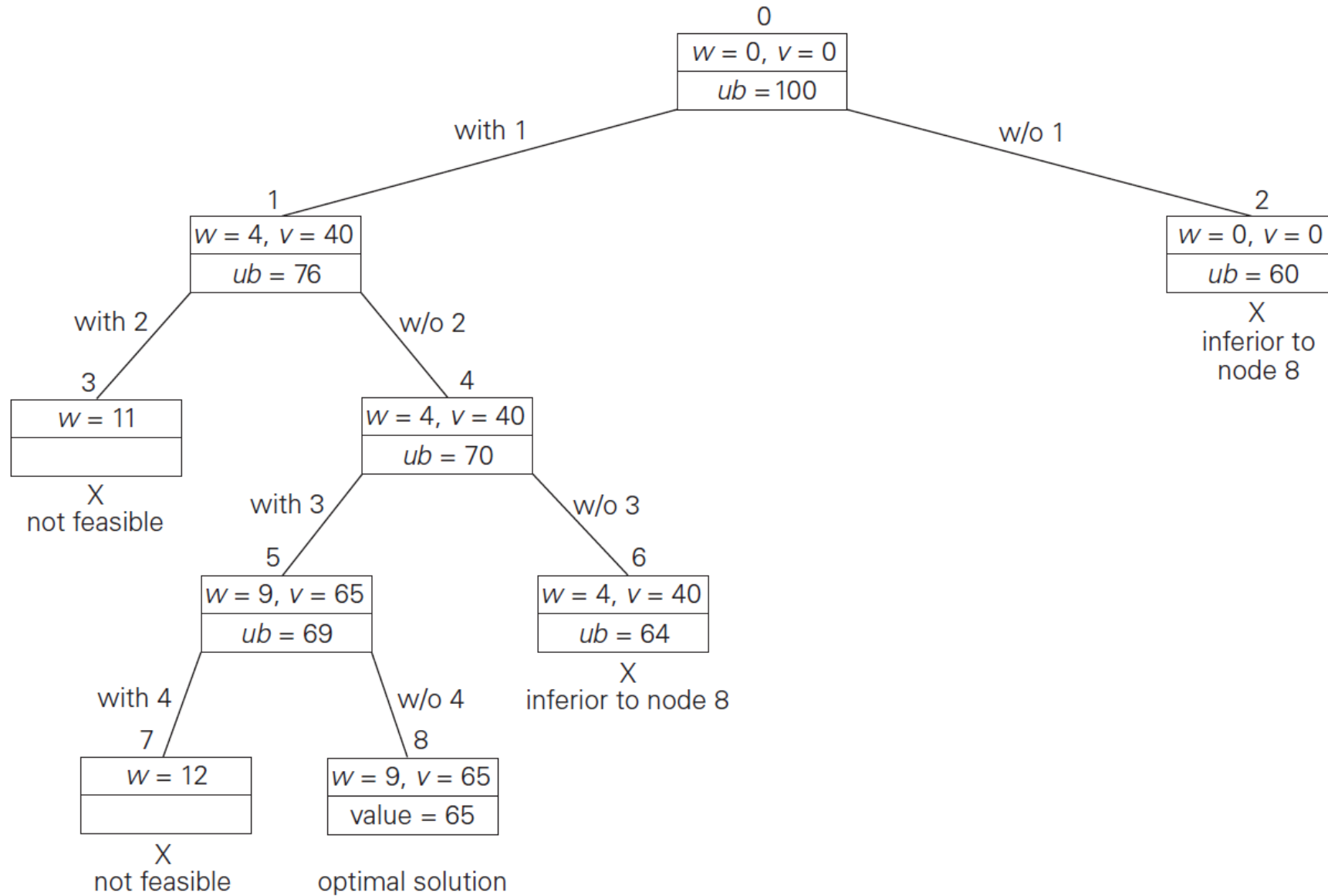
2. Solve the same instance of the assignment problem as the one solved in the section by the best-first branch-and-bound algorithm with the bounding function based on matrix columns rather than rows.
 3.
 - a. Give an example of the best-case input for the branch-and-bound algorithm for the assignment problem.
 - b. In the best case, how many nodes will be in the state-space tree of the branch-and-bound algorithm for the assignment problem?
- 

Knapsack Problem

item	weight	value	$\frac{\text{value}}{\text{weight}}$
1	4	\$40	10
2	7	\$42	6
3	5	\$25	5
4	3	\$12	4

The knapsack's capacity W is 10.

$$ub = v + (W - w)(v_{i+1}/w_{i+1}).$$

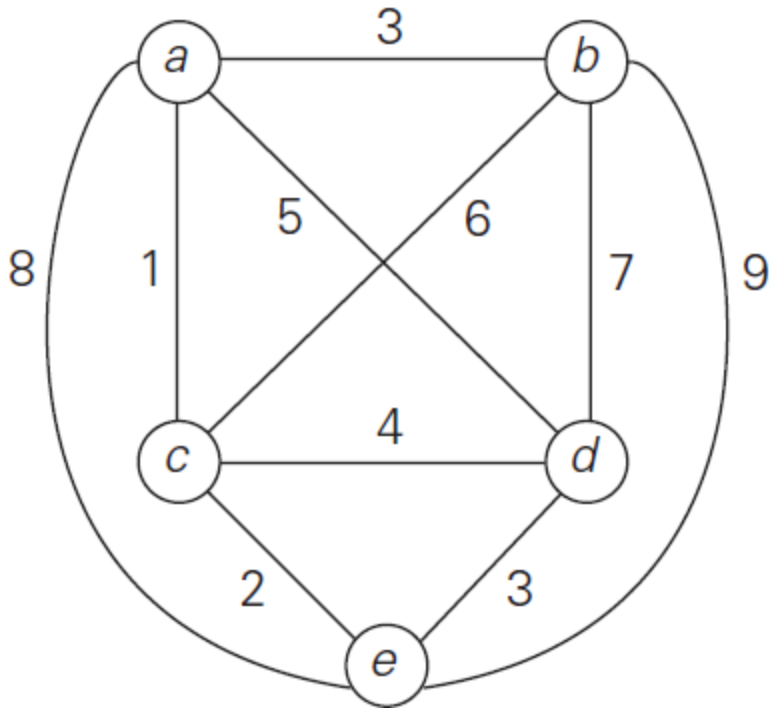


5. Solve the following instance of the knapsack problem by the branch-and-bound algorithm:

item	weight	value	$W = 16$
1	10	\$100	
2	7	\$63	
3	8	\$56	
4	4	\$12	

Travelling Salesperson Problem

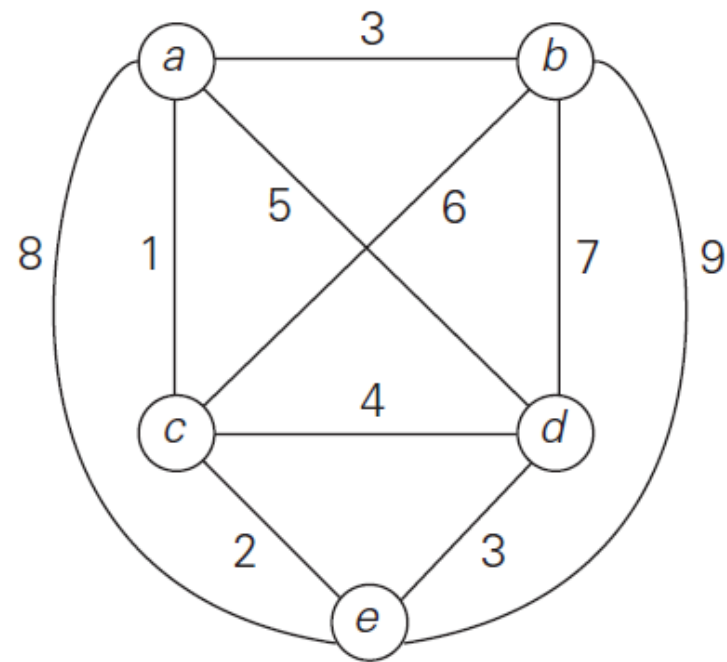
$$lb = \lceil s/2 \rceil.$$



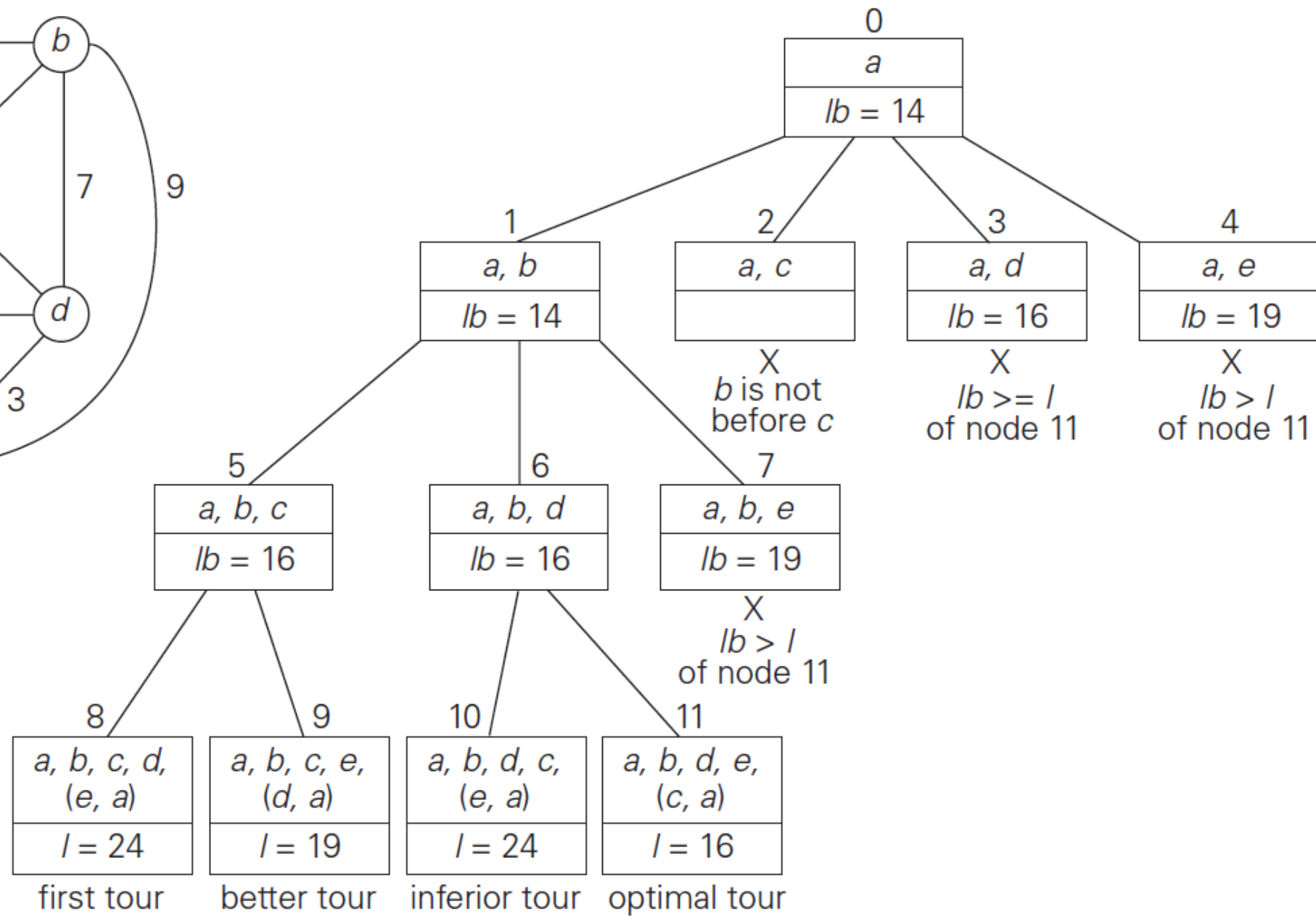
$$lb = \lceil [(1 + 3) + (3 + 6) + (1 + 2) + (3 + 4) + (2 + 3)]/2 \rceil = 14.$$

Moreover, for any subset of tours that must include particular edges of a given graph, we can modify lower bound (12.2) accordingly. For example, for all the Hamiltonian circuits of the graph in Figure 12.9a that must include edge (a, d) , we get the following lower bound by summing up the lengths of the two shortest edges incident with each of the vertices, with the required inclusion of edges (a, d) and (d, a)

$$\lceil [(1 + 5) + (3 + 6) + (1 + 2) + (3 + 5) + (2 + 3)]/2 \rceil = 16.$$



(a)



Optimizing Pharmaceutical Shipments

MediQuick Logistics "Prioritizing high-value vaccines, Bridging Urban and Rural Care."

A refrigerated truck with a **10,000 kg weight limit** must deliver high-priority vaccines and medicines to hospitals. Each product has a **weight**, **value** (priority score), and **region**. You are asked to select the optimal cargo to maximize life-saving impact without exceeding the weight limit keeping the constraints in mind from the boss.

Constraints:

1. Weight limit is 10,000 kg.
2. At least 1 product must go to Urban and 1 to Rural.
3. Ship entire products (no partial shipments).

Product	Weight (kg)	Value (Priority)	Region
Vaccine A	2000	95	Urban
Vaccine B	1500	80	Rural
Insulin	3000	90	Urban
Antibiotics	2500	70	Rural
PPE Kits	1000	60	Urban



Dynamic Programming

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- (Programming= in this context refers to a tabular method, not to writing computer code.)
- A dynamic-programming algorithm solves each subproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subproblem.
- Dynamic programming typically applies to *optimization problems*.
 - Such problems can have many possible solutions.
 - Each solution has a value, and you want to find a solution with the optimal (minimum or maximum) value.
 - We call such a solution *an* optimal solution to the problem
- As opposed to *the* optimal solution, since there may be several solutions that achieve the optimal value.
- To develop a dynamic-programming algorithm, follow a sequence of four steps:
 1. Characterize the structure of an optimal solution.
 2. Recursively define the value of an optimal solution.
 3. Compute the value of an optimal solution, typically in a bottom-up fashion.
 4. Construct an optimal solution from computed information.

Fibonacci numbers

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... ,

$$F(n) = F(n - 1) + F(n - 2) \quad \text{for } n > 1$$

$$F(0) = 0, \quad F(1) = 1.$$

EXAMPLE 1 *Coin-row problem* There is a row of n coins whose values are some positive integers $c_1, c_2, \dots, c_n$, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.

$$F(n) = \max\{c_n + F(n - 2), F(n - 1)\} \quad \text{for } n > 1,$$
$$F(0) = 0, \quad F(1) = c_1.$$

ALGORITHM *CoinRow*($C[1..n]$)

```
//Applies formula (8.3) bottom up to find the maximum amount of money
//that can be picked up from a coin row without picking two adjacent coins
//Input: Array  $C[1..n]$  of positive integers indicating the coin values
//Output: The maximum amount of money that can be picked up
 $F[0] \leftarrow 0; \quad F[1] \leftarrow C[1]$ 
for  $i \leftarrow 2$  to  $n$  do
     $F[i] \leftarrow \max(C[i] + F[i - 2], F[i - 1])$ 
return  $F[n]$ 
```

Solving the coin-row problem by dynamic programming for the coin row
5, 1, 2, 10, 6, 2.

$$F[0] = 0, F[1] = c_1 = 5$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5					

$$F[4] = \max\{10 + 5, 7\} = 15$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15		

$$F[2] = \max\{1 + 0, 5\} = 5$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5				

$$F[5] = \max\{6 + 7, 15\} = 15$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	

$$F[3] = \max\{2 + 5, 5\} = 7$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7			

$$F[6] = \max\{2 + 15, 15\} = 17$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	17

EXAMPLE 2 *Change-making problem* Consider the general instance of the following well-known problem. Give change for amount n using the minimum number of coins of denominations $d_1 < d_2 < \dots < d_m$. For the coin denominations used in the United States, as for those used in most if not all other countries, there is a very simple and efficient algorithm discussed in the next chapter. Here, we consider a dynamic programming algorithm for the general case, assuming availability of unlimited quantities of coins for each of the m denominations $d_1 < d_2 < \dots < d_m$ where $d_1 = 1$.

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

ALGORITHM *ChangeMaking*($D[1..m], n$)

//Applies dynamic programming to find the minimum number of coins
 //of denominations $d_1 < d_2 < \dots < d_m$ where $d_1 = 1$ that add up to a
 //given amount n

//Input: Positive integer n and array $D[1..m]$ of increasing positive
 //integers indicating the coin denominations where $D[1] = 1$

//Output: The minimum number of coins that add up to n

$F[0] \leftarrow 0$

for $i \leftarrow 1$ **to** n **do**

$temp \leftarrow \infty$; $j \leftarrow 1$

while $j \leq m$ **and** $i \geq D[j]$ **do**

$temp \leftarrow \min(F[i - D[j]], temp)$

$j \leftarrow j + 1$

$F[i] \leftarrow temp + 1$

return $F[n]$

Application of Algorithm *MinCoinChange* to amount $n = 6$ and coin denominations 1, 3, and 4.

$$F[0] = 0$$

n	0	1	2	3	4	5	6
F	0						

$$F[1] = \min\{F[1 - 1]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1					

$$F[2] = \min\{F[2 - 1]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2				

$$F[3] = \min\{F[3 - 1], F[3 - 3]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1	2	1			

$$F[4] = \min\{F[4 - 1], F[4 - 3], F[4 - 4]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1		

$$F[5] = \min\{F[5 - 1], F[5 - 3], F[5 - 4]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	

$$F[6] = \min\{F[6 - 1], F[6 - 3], F[6 - 4]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2

EXAMPLE 3 *Coin-collecting problem* Several coins are placed in cells of an $n \times m$ board, no more than one coin per cell. A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell. On each step, the robot can move either one cell to the right or one cell down from its current location. When the robot visits a cell with a coin, it always picks up that coin. Design an algorithm to find the maximum number of coins the robot can collect and a path it needs to follow to do this.

$$F(i, j) = \max\{F(i - 1, j), F(i, j - 1)\} + c_{ij} \quad \text{for } 1 \leq i \leq n, \quad 1 \leq j \leq m$$

ALGORITHM *RobotCoinCollection*($C[1..n, 1..m]$) $F(0, j) = 0$ for $1 \leq j \leq m$ and $F(i, 0) = 0$ for $1 \leq i \leq n$,

//Applies dynamic programming to compute the largest number of

//coins a robot can collect on an $n \times m$ board by starting at (1, 1)

//and moving right and down from upper left to down right corner

//Input: Matrix $C[1..n, 1..m]$ whose elements are equal to 1 and 0

//for cells with and without a coin, respectively

//Output: Largest number of coins the robot can bring to cell (n, m)

$F[1, 1] \leftarrow C[1, 1];$ **for** $j \leftarrow 2$ **to** m **do** $F[1, j] \leftarrow F[1, j - 1] + C[1, j]$

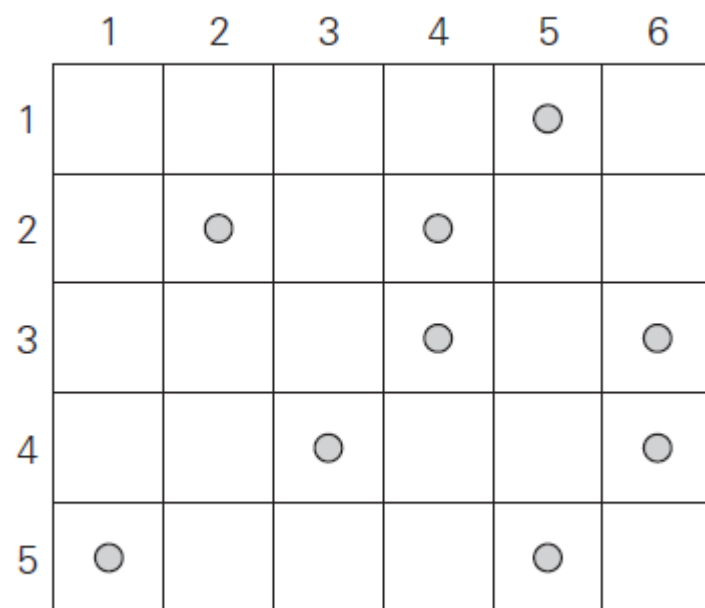
for $i \leftarrow 2$ **to** n **do**

$F[i, 1] \leftarrow F[i - 1, 1] + C[i, 1]$

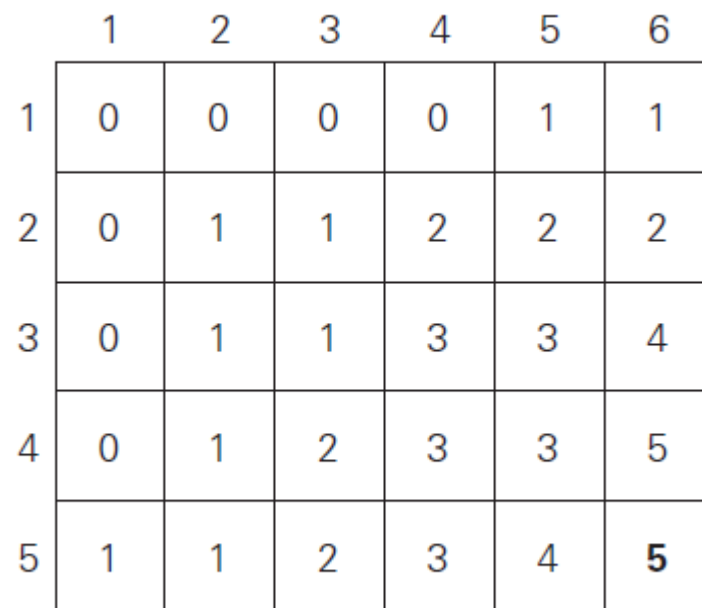
for $j \leftarrow 2$ **to** m **do**

$F[i, j] \leftarrow \max(F[i - 1, j], F[i, j - 1]) + C[i, j]$

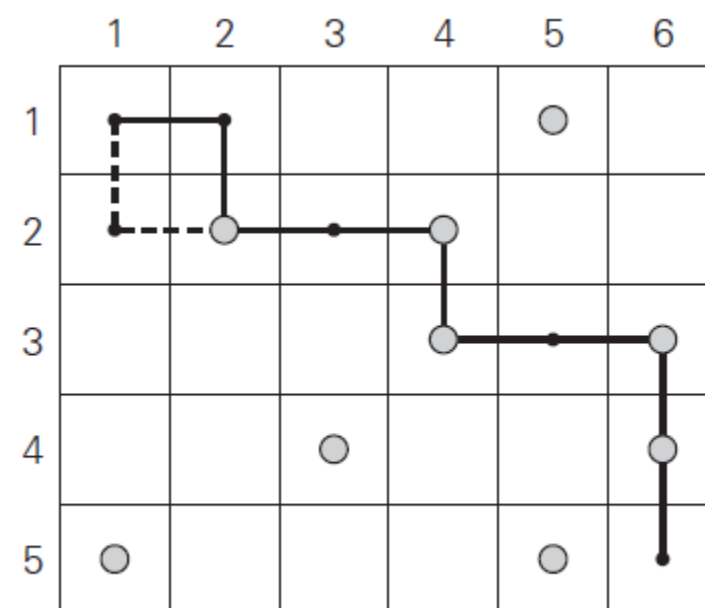
return $F[n, m]$



(a)



(b)



(c)

(a) Coins to collect. (b) Dynamic programming algorithm results. (c) Two paths to collect 5 coins, the maximum number of coins possible.

To solve the original problem of size n , you solve smaller problems of the same type. Once you make the first cut, the two resulting pieces form independent instances of the rod-cutting problem. The overall optimal solution incorporates optimal solutions to the two resulting subproblems, maximizing revenue from each of those two pieces. We say that the rod-cutting problem exhibits *optimal substructure*: optimal solutions to a problem incorporate optimal solutions to related subproblems, which you may solve independently.

The *rod-cutting problem* is the following. Given a rod of length n inches and a table of prices p_i for $i = 1, 2, \dots, n$, determine the maximum revenue r_n obtainable by cutting up the rod and selling the pieces. If the price p_n for a rod of length n is large enough, an optimal solution might require no cutting at all.

$$r_n = \max \{ p_i + r_{n-i} : 1 \leq i \leq n \} \quad \text{or} \quad r_n = \max \{ p_i + r_1 : 1 \leq i \leq n \} .$$

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

$r_1 = 1$ from solution 1 = 1 (no cuts), $r_6 = 17$ from solution 6 = 6 (no cuts),
 $r_2 = 5$ from solution 2 = 2 (no cuts), $r_7 = 18$ from solution 7 = 1 + 6 or 7 = 2 + 2 + 3,
 $r_3 = 8$ from solution 3 = 3 (no cuts), $r_8 = 22$ from solution 8 = 2 + 6,
 $r_4 = 10$ from solution 4 = 2 + 2, $r_9 = 25$ from solution 9 = 3 + 6,
 $r_5 = 13$ from solution 5 = 2 + 3, $r_{10} = 30$ from solution 10 = 10 (no cuts).

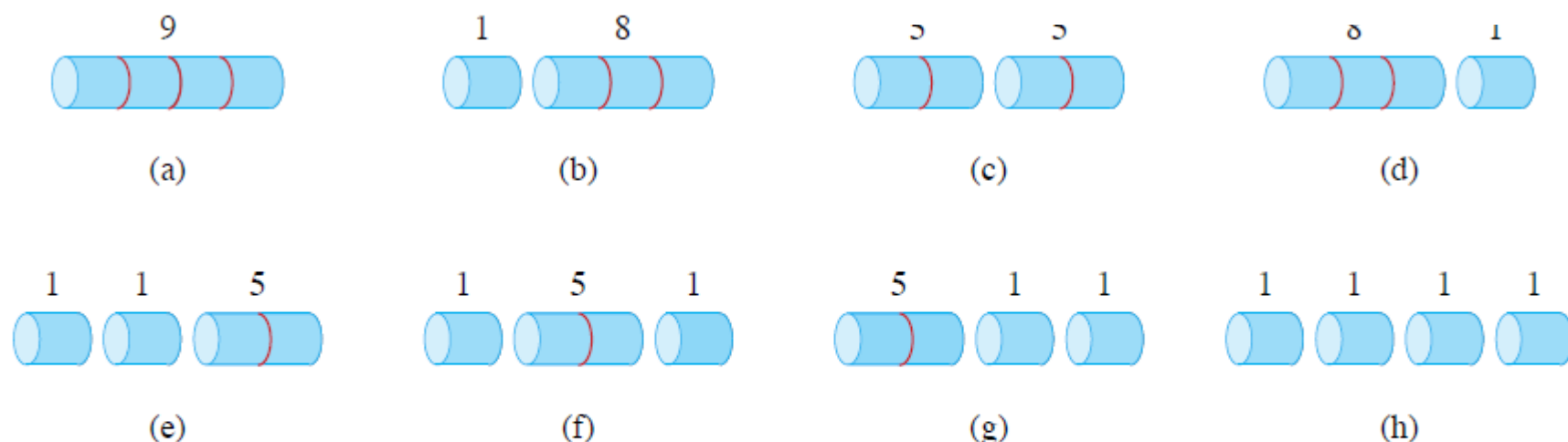
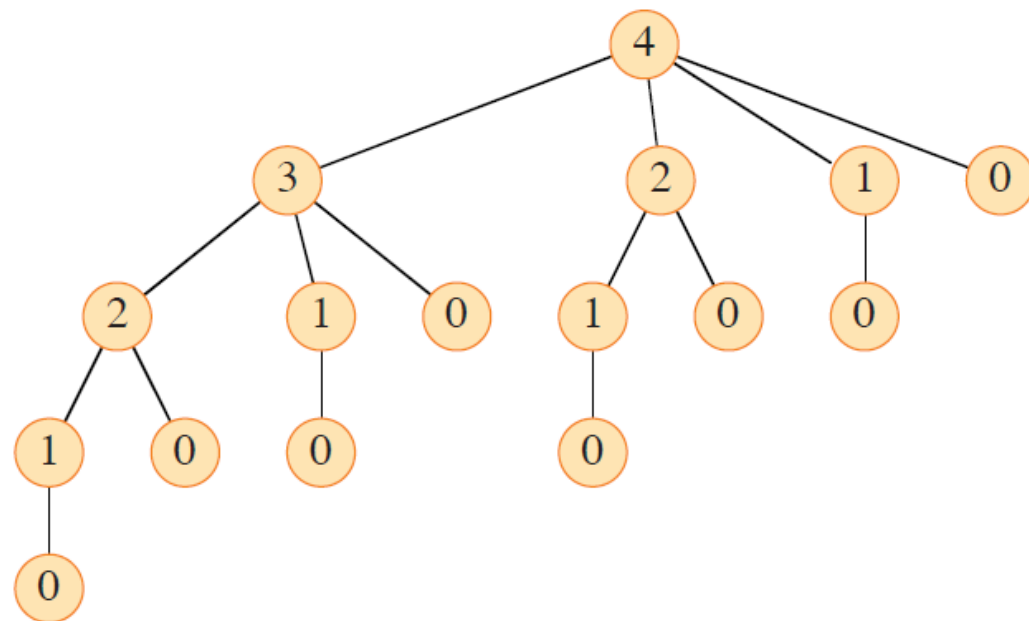


Figure 14.2 The 8 possible ways of cutting up a rod of length 4. Above each piece is the value of that piece, according to the sample price chart of Figure 14.1. The optimal strategy is part (c)—cutting the rod into two pieces of length 2—which has total value 10.

Recursive top-down implementation

CUT-ROD(p, n)

```
1  if  $n == 0$ 
2      return 0
3   $q = -\infty$ 
4  for  $i = 1$  to  $n$ 
5       $q = \max \{q, p[i] + \text{CUT-ROD}(p, n - i)\}$ 
6  return  $q$ 
```



To analyze the running time of CUT-ROD, let $T(n)$ denote the total number of calls made to CUT-ROD(p, n) for a particular value of n . This expression equals the number of nodes in a subtree whose root is labeled n in the recursion tree. The count includes the initial call at its root. Thus, $T(0) = 1$ and

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j).$$

The initial 1 is for the call at the root, and the term $T(j)$ counts the number of calls (including recursive calls) due to the call CUT-ROD($p, n - i$), where $j = n - i$. As Exercise 14.1-1 asks you to show,

$$T(n) = 2^n,$$

CUT-ROD is exponential in n .

Using dynamic programming for optimal rod cutting

The dynamic-programming method works as follows. Instead of solving the same subproblems repeatedly, as in the naive recursion solution, arrange for each subproblem to be solved *only once*. There's actually an obvious way to do so: the first time you solve a subproblem, *save its solution*. If you need to refer to this subproblem's solution again later, just look it up, rather than recomputing it.

- Saving subproblem solutions comes with a cost: the additional memory needed to store solutions. Dynamic programming thus serves as an example of a time-memory trade-off.
- A dynamic-programming approach runs in polynomial time when the number of distinct subproblems involved is polynomial in the input size and you can solve each such subproblem in polynomial time.

The first approach is *top-down* with *memoization*.² In this approach, you write the procedure recursively in a natural manner, but modified to save the result of each subproblem (usually in an array or hash table). The procedure now first checks to see whether it has previously solved this subproblem. If so, it returns the saved value, saving further computation at this level. If not, the procedure computes the value in the usual manner but also saves it. We say that the recursive procedure has been *memoized*: it “remembers” what results it has computed previously.

The second approach is the *bottom-up method*. This approach typically depends on some natural notion of the “size” of a subproblem, such that solving any particular subproblem depends only on solving “smaller” subproblems. Solve the subproblems in size order, smallest first, storing the solution to each subproblem when it is first solved. In this way, when solving a particular subproblem, there are already saved solutions for all of the smaller subproblems its solution depends upon. You need to solve each subproblem only once, and when you first see it, you have already solved all of its prerequisite subproblems.

These two approaches yield algorithms with the same asymptotic running time, except in unusual circumstances where the **top-down approach does not actually recurse to examine all possible subproblems**. The **bottom-up approach often has much better constant factors, since it has lower overhead for procedure calls**.

MEMOIZED-CUT-ROD(p, n)

```
1  let  $r[0:n]$  be a new array      // will remember solution values in  $r$ 
2  for  $i = 0$  to  $n$ 
3       $r[i] = -\infty$ 
4  return MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
```

MEMOIZED-CUT-ROD-AUX(p, n, r)

```
1  if  $r[n] \geq 0$                   // already have a solution for length  $n$ ?
2      return  $r[n]$ 
3  if  $n == 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$           //  $i$  is the position of the first cut
7           $q = \max\{q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r)\}$ 
8   $r[n] = q$                     // remember the solution value for length  $n$ 
9  return  $q$ 
```

BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0:n]$  be a new array           // will remember solution values in  $r$ 
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$                        // for increasing rod length  $j$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$                    //  $i$  is the position of the first cut
6           $q = \max\{q, p[i] + r[j - i]\}$ 
7       $r[j] = q$                          // remember the solution value for length  $j$ 
8  return  $r[n]$ 
```

The Knapsack Problem and Memory Functions

Our goal is to find $F(n, W)$, the maximal value of a subset of the n given items that fit into the knapsack of capacity W , and an optimal subset itself.

Let us consider an instance defined by the first i items, $1 \leq i \leq n$, with weights $w_1, \dots, w_i$, values $v_1, \dots, v_i$, and knapsack capacity j , $1 \leq j \leq W$. Let $F(i, j)$ be the value of an optimal solution to this instance, i.e., the value of the most valuable subset of the first i items that fit into the knapsack of capacity j . We can divide all the subsets of the first i items that fit into the knapsack of capacity j into two categories: those that do not include the i th item and those that do. Note the following:

1. Among the subsets that do not include the i th item, the value of an optimal subset is, by definition, $F(i - 1, j)$.
2. Among the subsets that do include the i th item (hence, $j - w_i \geq 0$), an optimal subset is made up of this item and an optimal subset of the first $i - 1$ items that fits into the knapsack of capacity $j - w_i$. The value of such an optimal

$$F(i, j) = \begin{cases} \max\{F(i - 1, j), v_i + F(i - 1, j - w_i)\} & \text{if } j - w_i \geq 0, \\ F(i - 1, j) & \text{if } j - w_i < 0. \end{cases}$$

$$F(0, j) = 0 \text{ for } j \geq 0 \quad \text{and} \quad F(i, 0) = 0 \text{ for } i \geq 0.$$

EXAMPLE 1 Let us consider the instance given by the following data:

item	weight	value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

capacity $W = 5$.

		capacity j						
		i	0	1	2	3	4	5
		0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	0	12	12	12	12	12
$w_2 = 1, v_2 = 10$	2	0	10	12	22	22	22	22
$w_3 = 3, v_3 = 20$	3	0	10	12	22	30	32	32
$w_4 = 2, v_4 = 15$	4	0	10	15	25	30	37	

FIGURE 8.5 Example of solving an instance of the knapsack problem by the dynamic programming algorithm.

ALGORITHM $MFKnapsack(i, j)$

//Implements the memory function method for the knapsack problem
//Input: A nonnegative integer i indicating the number of the first
// items being considered and a nonnegative integer j indicating
// the knapsack capacity
//Output: The value of an optimal feasible subset of the first i items
//Note: Uses as global variables input arrays $Weights[1..n]$, $Values[1..n]$,
//and table $F[0..n, 0..W]$ whose entries are initialized with -1 's except for
//row 0 and column 0 initialized with 0's
if $F[i, j] < 0$
 if $j < Weights[i]$
 $value \leftarrow MFKnapsack(i - 1, j)$
 else
 $value \leftarrow \max(MFKnapsack(i - 1, j),$
 $Values[i] + MFKnapsack(i - 1, j - Weights[i]))$
 $F[i, j] \leftarrow value$
return $F[i, j]$

		capacity j						
		i	0	1	2	3	4	5
		0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	0	12	12	12	12	12
$w_2 = 1, v_2 = 10$	2	0	—	12	22	—	22	22
$w_3 = 3, v_3 = 20$	3	0	—	—	22	—	32	32
$w_4 = 2, v_4 = 15$	4	0	—	—	—	—	—	37

FIGURE 8.6 Example of solving an instance of the knapsack problem by the memory function algorithm.

Matrix-chain multiplication

RECTANGULAR-MATRIX-MULTIPLY(A, B, C, p, q, r)

```
1  for  $i = 1$  to  $p$ 
2      for  $j = 1$  to  $r$ 
3          for  $k = 1$  to  $q$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
```

$(A_1(A_2(A_3A_4)))$,
 $(A_1((A_2A_3)A_4))$,
 $((A_1A_2)(A_3A_4))$,
 $((A_1(A_2A_3))A_4)$,
 $((((A_1A_2)A_3)A_4))$.

We state the *matrix-chain multiplication problem* as follows: given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1A_2 \cdots A_n$ in a way that minimizes the number of scalar multiplications. The input is the sequence of dimensions

$$m[i, j] = \begin{cases} 0 & \text{if } i = j , \\ \min \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j : i \leq k < j\} & \text{if } i < j . \end{cases}$$

running time of $O(n^3)$

$\Theta(n^2)$ space to store the m and s tables.

MATRIX-CHAIN-ORDER(p, n)

```
1  let  $m[1:n, 1:n]$  and  $s[1:n-1, 2:n]$  be new tables
2  for  $i = 1$  to  $n$                                 // chain length 1
3       $m[i, i] = 0$ 
4  for  $l = 2$  to  $n$                                 //  $l$  is the chain length
5      for  $i = 1$  to  $n - l + 1$                     // chain begins at  $A_i$ 
6           $j = i + l - 1$                         // chain ends at  $A_j$ 
7           $m[i, j] = \infty$ 
8          for  $k = i$  to  $j - 1$                     // try  $A_{i:k} A_{k+1:j}$ 
9               $q = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$ 
10             if  $q < m[i, j]$ 
11                  $m[i, j] = q$                     // remember this cost
12                  $s[i, j] = k$                     // remember this index
13  return  $m$  and  $s$ 
```

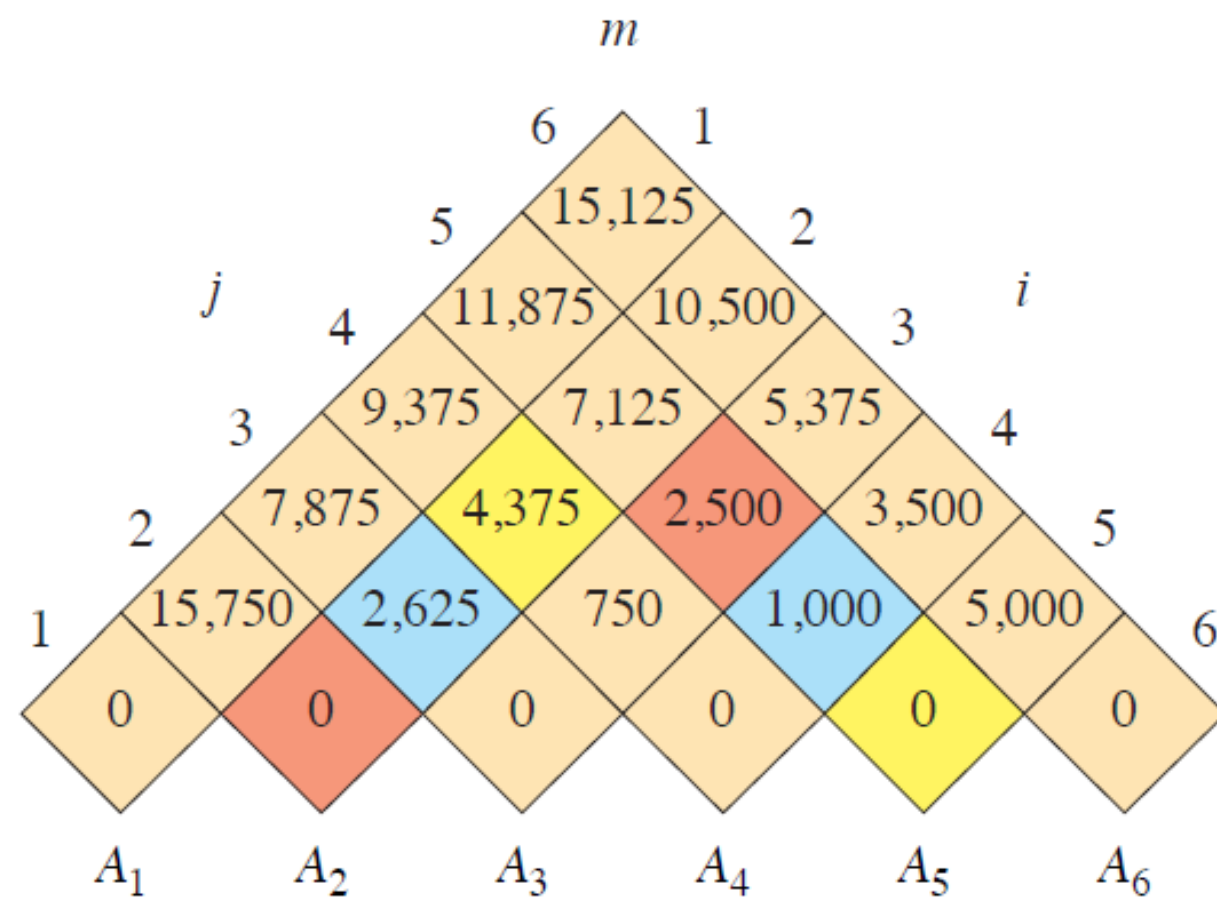


Figure 14.5 The m and s tables computed by MATRIX-CHAIN-ORDER for $n = 6$ and the following matrix dimensions:

matrix	A_1	A_2	A_3	A_4	A_5	A_6
dimension	30×35	35×15	15×5	5×10	10×20	20×25

Greedy Technique

A **greedy algorithm** always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice leads to a globally optimal solution

The first key ingredient is the **greedy-choice property**: you can assemble a globally optimal solution by making locally optimal (greedy) choices. In other words, when you are considering which choice to make, you make the choice that looks best in the current problem, without considering results from subproblems.

- **Dynamic programming, which solves the subproblems before making the first**
- **Greedy algorithm makes its first choice before solving any subproblems.**

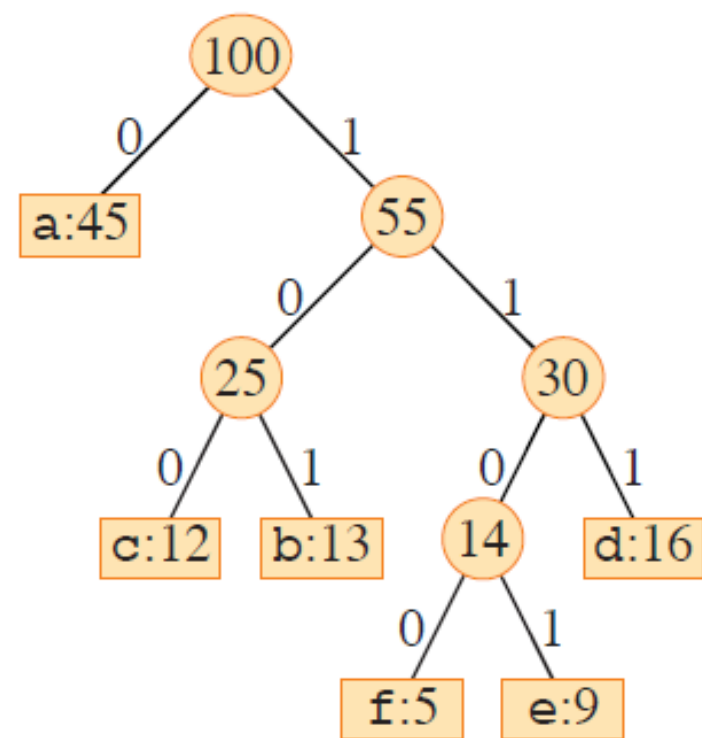
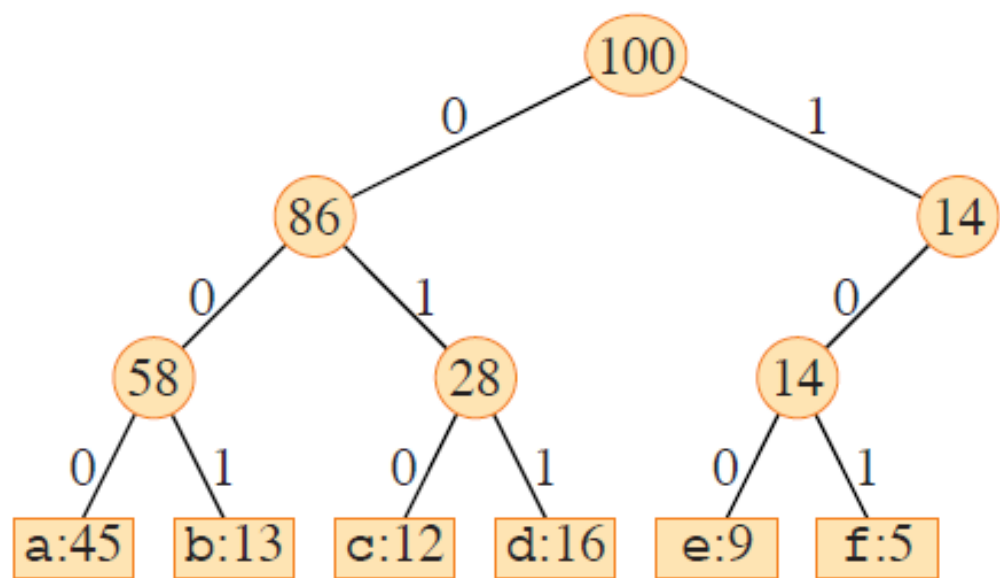
Huffman codes

Huffman codes compress data well: savings of 20% to 90% are typical, depending on the characteristics of the data being compressed. The data arrive as a sequence of characters. Huffman's greedy algorithm uses a table giving how often each character occurs (its frequency) to build up an optimal way of representing each character as a binary string.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

- If you use a **fixed-length code**, you need $\text{ceil}(\lg n)$ bits to represent $n \geq 2$ characters.
- A **variable-length code** can do considerably better than a fixed-length code. The idea is simple: give frequent characters short codewords and infrequent characters long codewords.

We consider here only codes in which no codeword is also a prefix of some other codeword. Such codes are called **prefix-free codes**

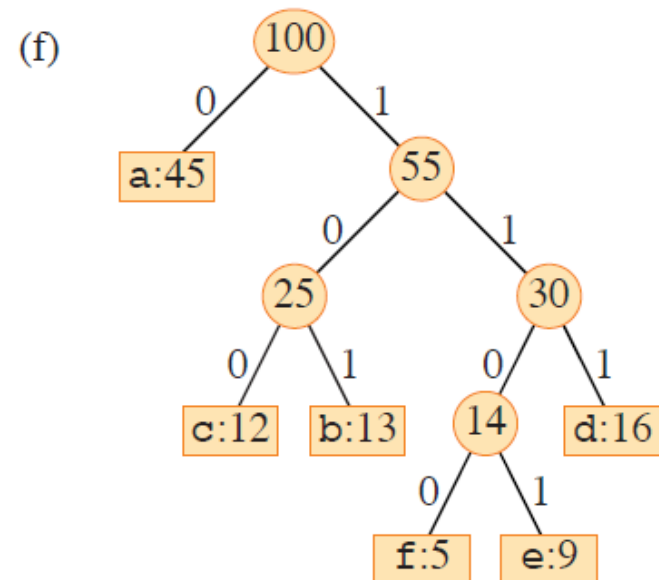
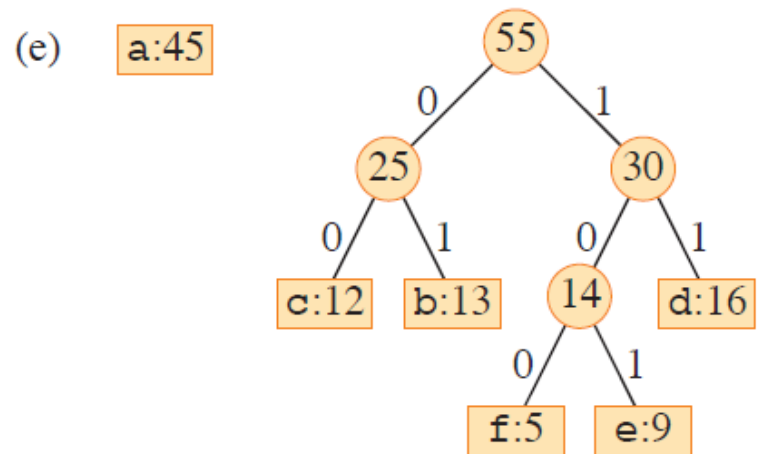
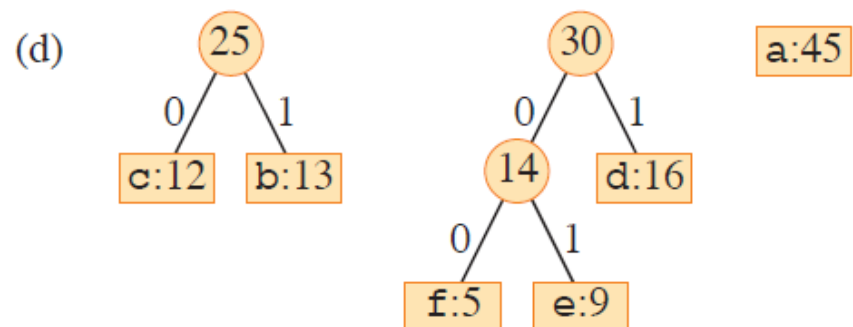
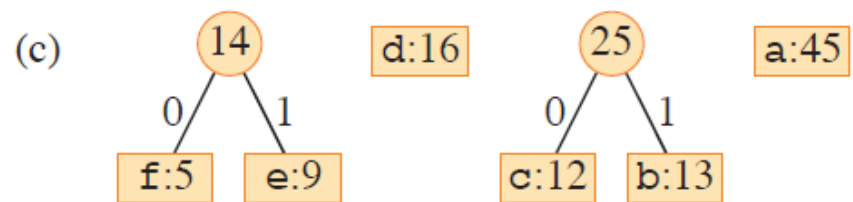
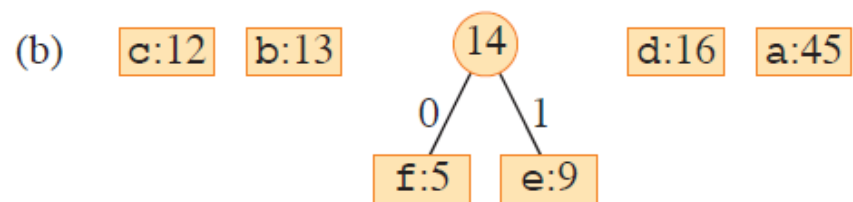


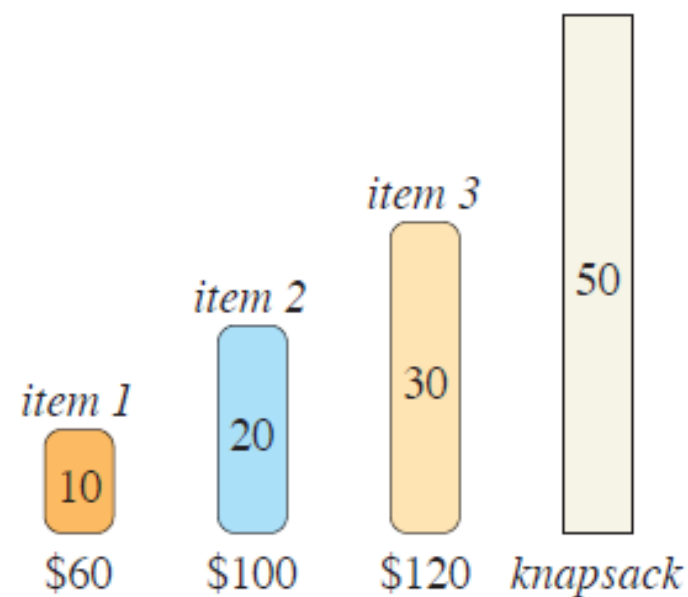
The running time of Huffman's algorithm depends on how the min-priority queue Q is implemented. Let's assume that it's implemented as a binary min-heap (see Chapter 6). For a set C of n characters, the BUILD-MIN-HEAP procedure discussed in Section 6.3 can initialize Q in line 2 in $O(n)$ time. The **for** loop in lines 3–10 executes exactly $n - 1$ times, and since each heap operation runs in $O(\lg n)$ time, the loop contributes $O(n \lg n)$ to the running time. Thus, the total running time of HUFFMAN on a set of n characters is $O(n \lg n)$.

HUFFMAN(C)

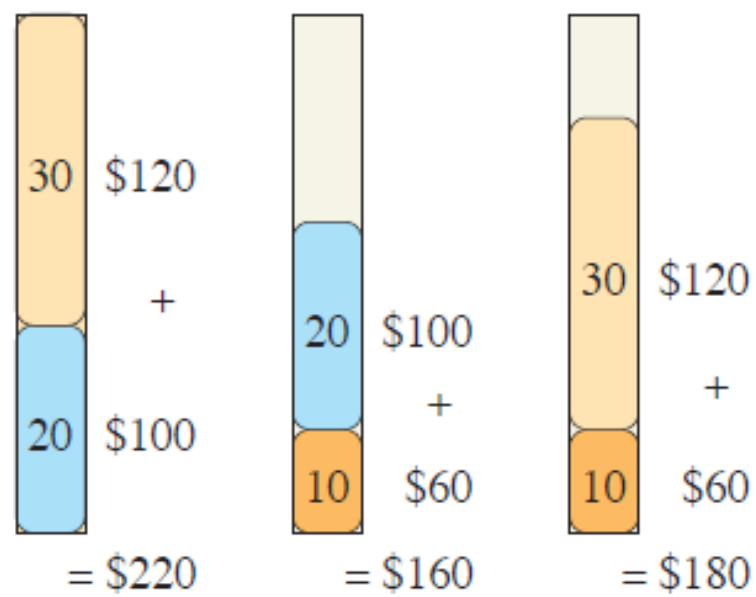
```
1   $n = |C|$ 
2   $Q = C$ 
3  for  $i = 1$  to  $n - 1$ 
4      allocate a new node  $z$ 
5       $x = \text{EXTRACT-MIN}(Q)$ 
6       $y = \text{EXTRACT-MIN}(Q)$ 
7       $z.\text{left} = x$ 
8       $z.\text{right} = y$ 
9       $z.\text{freq} = x.\text{freq} + y.\text{freq}$ 
10      $\text{INSERT}(Q, z)$ 
11 return  $\text{EXTRACT-MIN}(Q)$     // the root of the tree is the only node left
```

(a) f:5 e:9 c:12 b:13 d:16 a:45

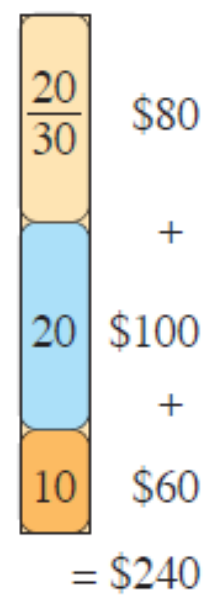




(a)



(b)

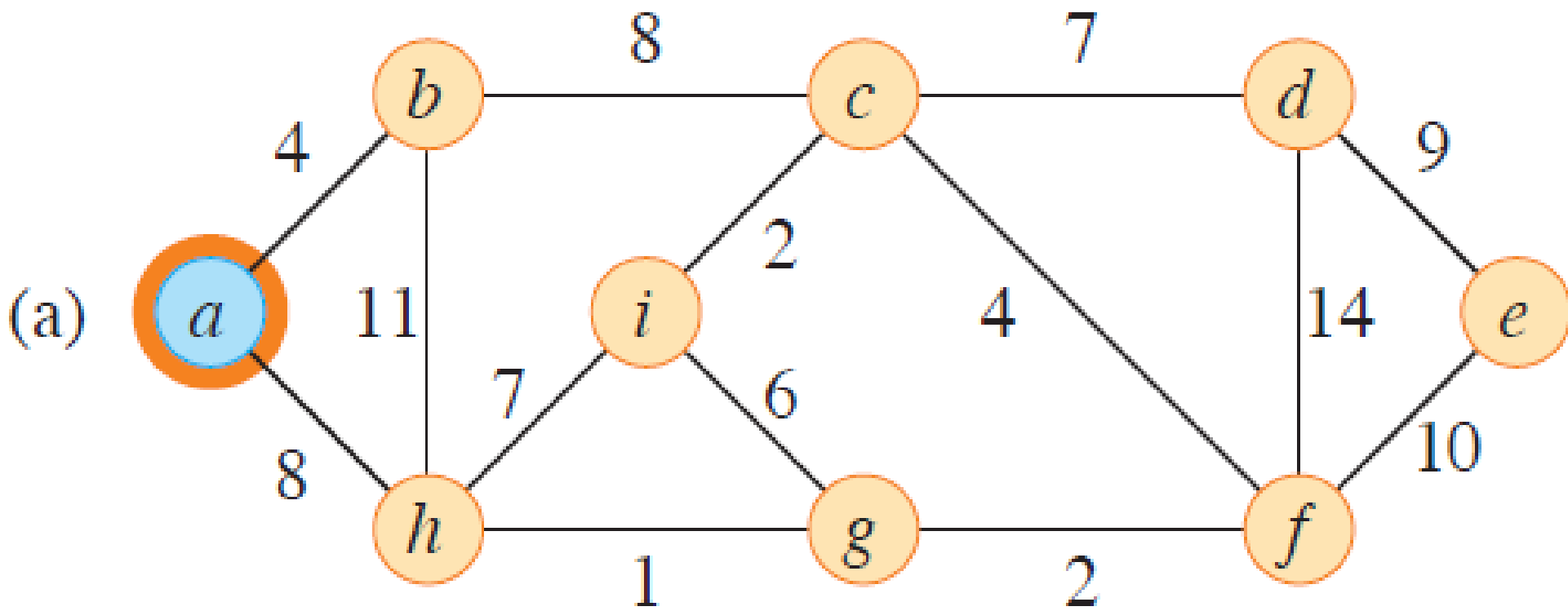


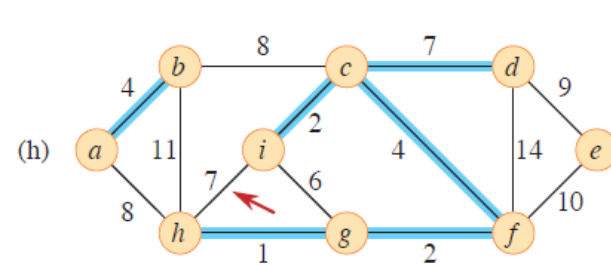
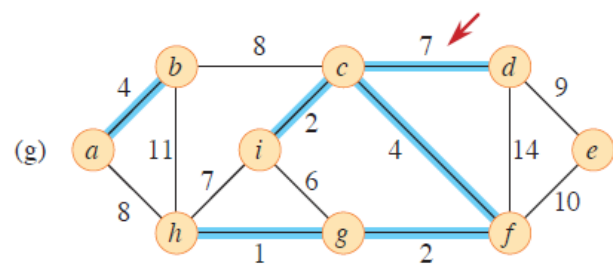
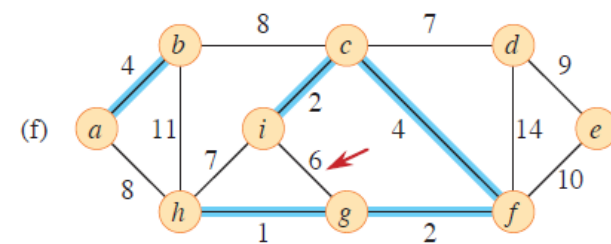
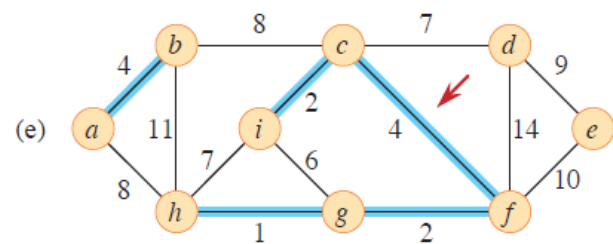
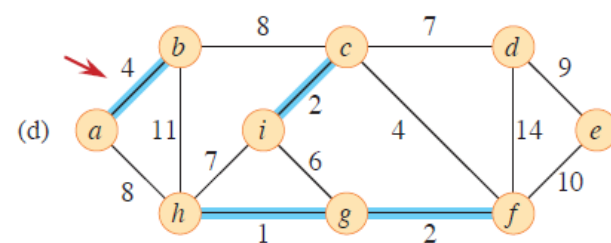
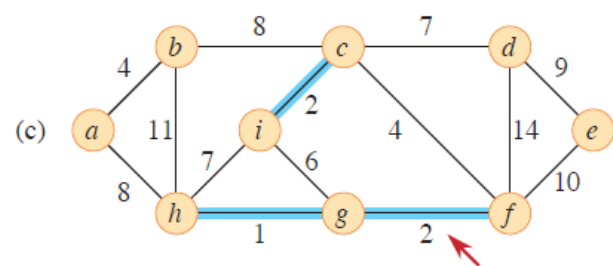
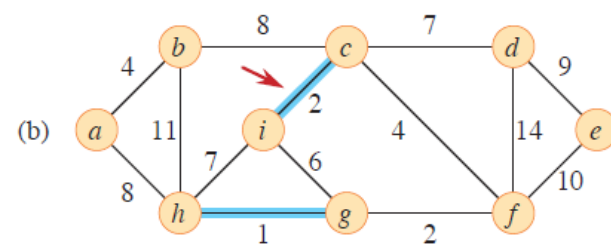
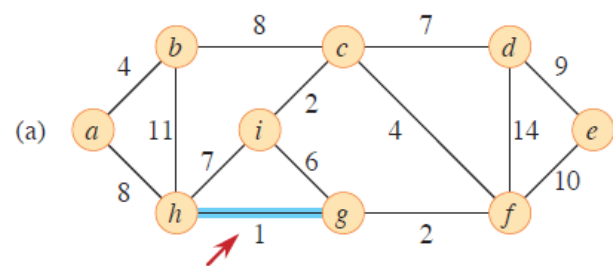
(c)

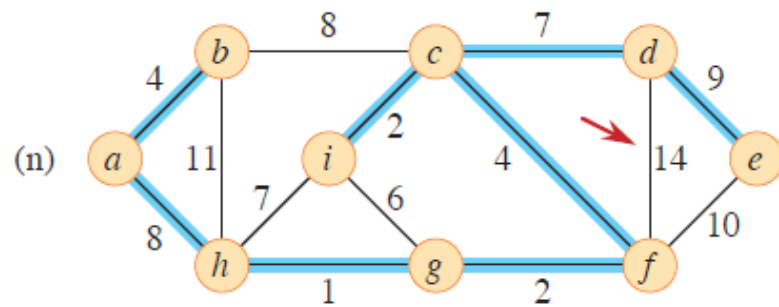
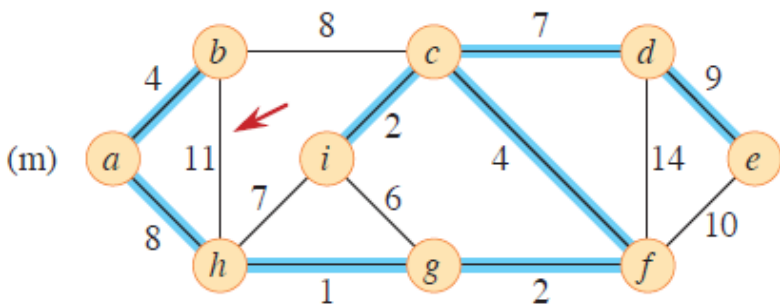
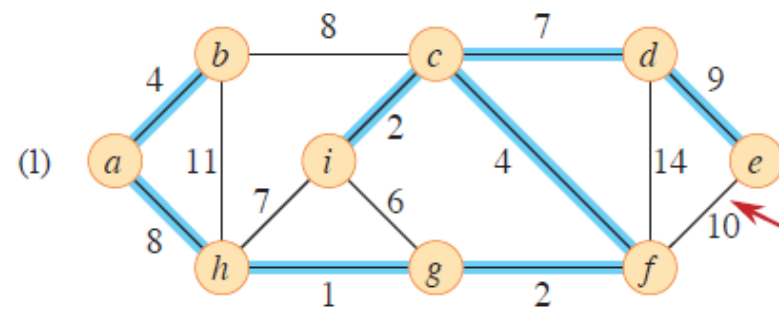
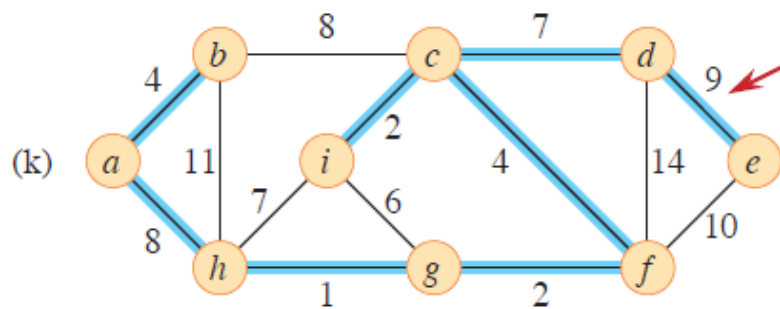
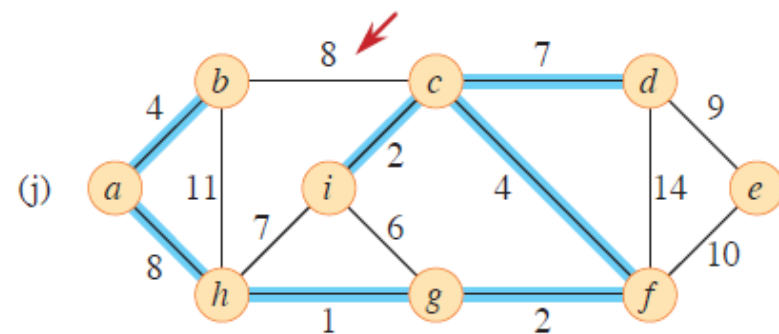
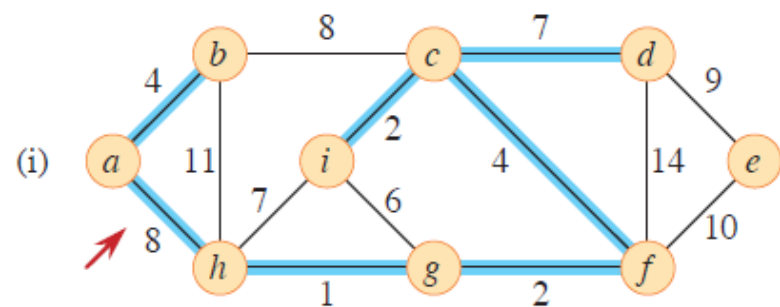
running time of Kruskal's algorithm as $O(E \lg V)$.

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  create a single list of the edges in  $G.E$ 
5  sort the list of edges into monotonically increasing order by weight  $w$ 
6  for each edge  $(u, v)$  taken from the sorted list in order
7      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
8           $A = A \cup \{(u, v)\}$ 
9          UNION( $u, v$ )
10 return  $A$ 
```

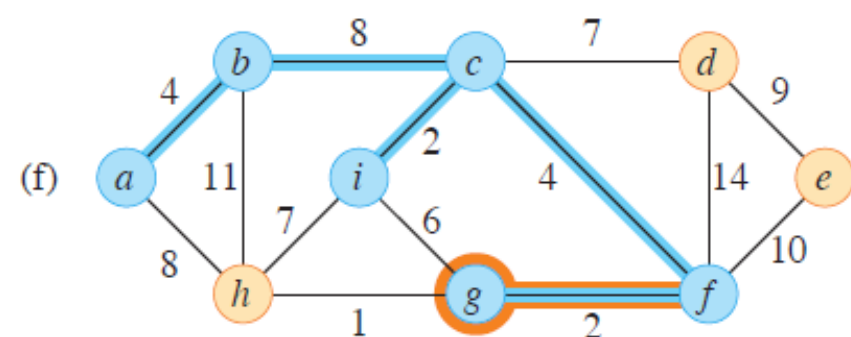
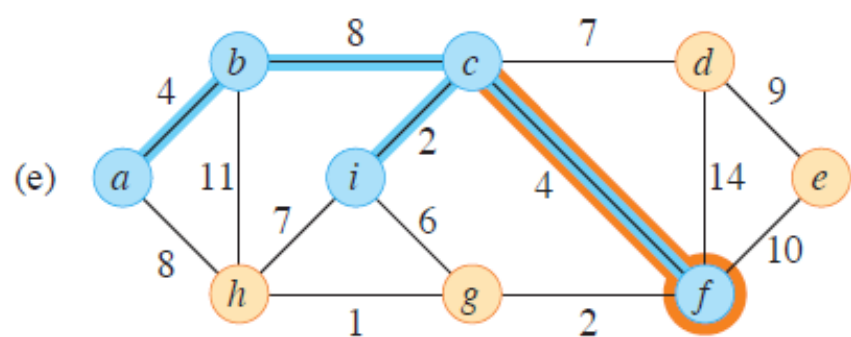
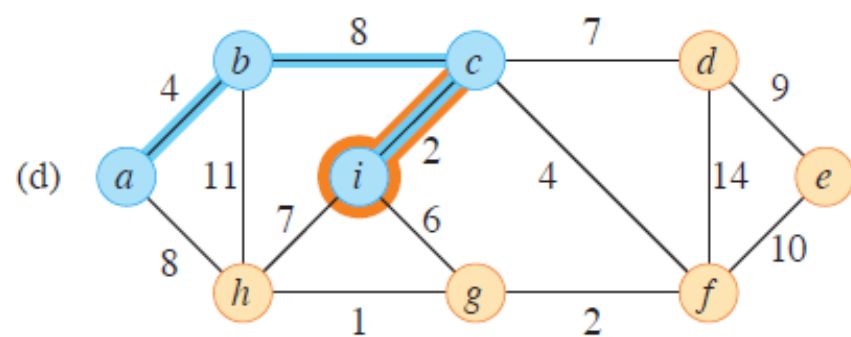
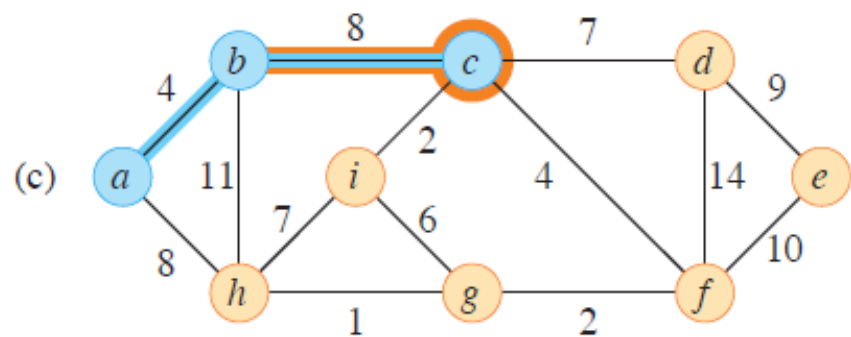
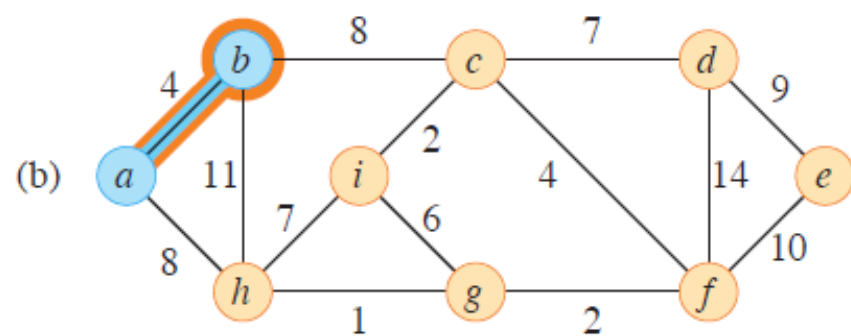
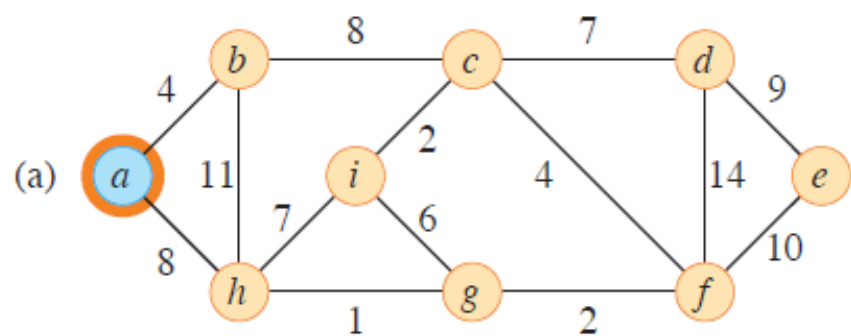


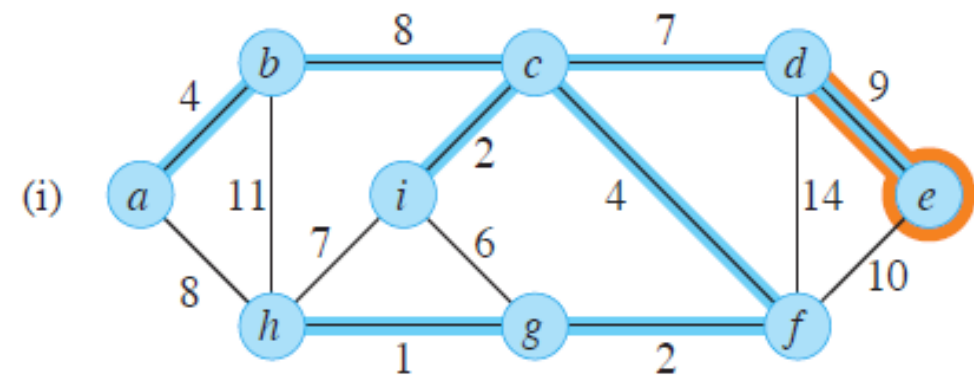
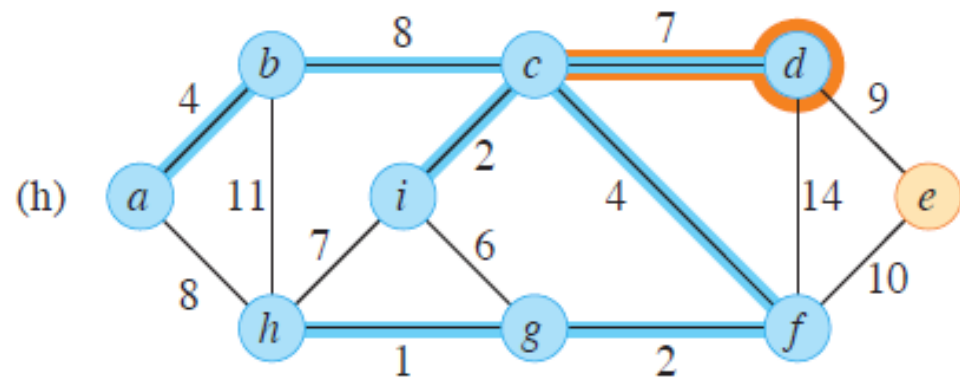
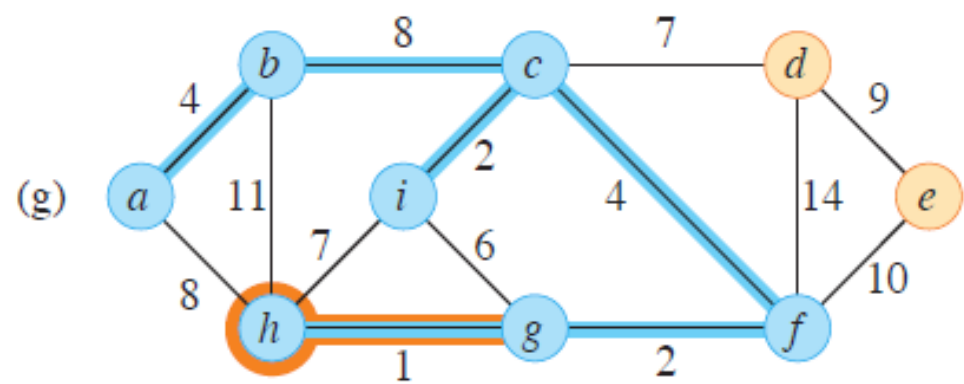


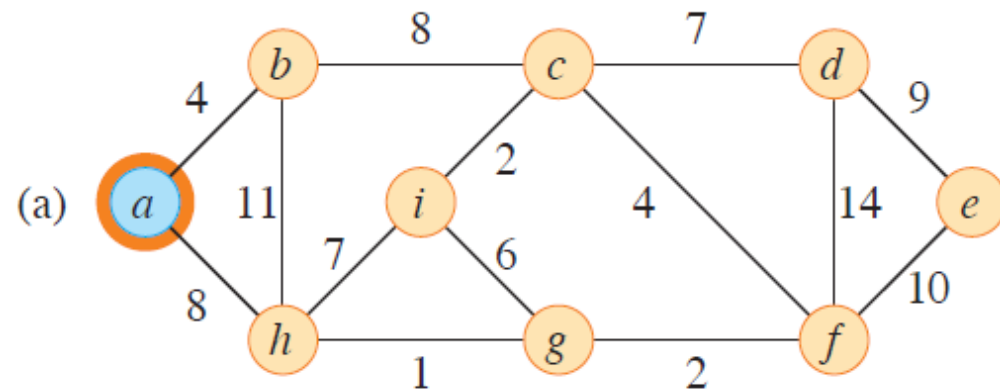


MST-PRIM(G, w, r)

```
1  for each vertex  $u \in G.V$ 
2       $u.key = \infty$   running time of Prim's algorithm improves to  $O(E + V \lg V)$ .
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = \emptyset$ 
6  for each vertex  $u \in G.V$ 
7      INSERT( $Q, u$ )
8  while  $Q \neq \emptyset$ 
9       $u = \text{EXTRACT-MIN}(Q)$       // add  $u$  to the tree
10     for each vertex  $v$  in  $G.Adj[u]$   // update keys of  $u$ 's non-tree neighbors
11         if  $v \in Q$  and  $w(u, v) < v.key$ 
12              $v.\pi = u$ 
13              $v.key = w(u, v)$ 
14             DECREASE-KEY( $Q, v, w(u, v)$ )
```







- Apply BFS and DFS to find the spanning tree of the graph.
- Change the source vertex to generate MST using Kruskal and Prim's.